DOKPANTAI LAMONGAN

PT Arunika Olah Pancarona

BUKU PELATIHAN PROFESIONAL

FLUTTER

ANDROID & iOS

Dari UI hingga Release

Praktikum • Evaluasi • Proyek Inspeksi

UI & FORM | REST API | STATE | TESTING

3 PROGRAM INTENSIF **HARI**

TRAINER & PENULIS

Damar Galih Aji Pradana, S.Tr.Kom.

PT. Dok Pantai Lamongan

Edisi Oktober 2026

Daftar isi

Tentang buku ini....5
Cara menggunakan buku....6
Peta perjalanan tiga hari....7
Kontrak studi kasus inspeksi....8
Prasyarat dan catatan versi....9
Ruang lingkup dan hasil akhir....10
01. Flutter sebagai UI deklaratif....11
02. Membuat dan menjalankan proyek....12
03. Mengenali struktur proyek....13
04. Hot reload, restart, dan full run....14
05. Tipe, final, dan const....15
06. Null safety dan fallback bermakna....16
07. Fungsi dan named parameter....17
08. Model Inspection yang konsisten....18
09. Collection, map, dan where....19
10. Future dan async/await....20
11. Praktikum model dan data lokal....21
12. Membaca widget tree....22
13. StatelessWidget dan StatefulWidget....23
14. Scaffold, SafeArea, dan aksi utama....24
15. Constraint dan ukuran tersedia....25
16. Row, Flexible, dan teks panjang....26
17. ListView.builder dan key record....27
18. Komponen reusable dan theme....28
19. Form scroll dan keyboard....29
20. Asset dan konfigurasi pubspec....30
21. Praktikum daftar dan detail....31
22. Lifecycle State dan resource....32
23. Loading, success, empty, dan error....33
24. Load yang aman terhadap lifecycle....34
25. Form dan validator field wajib....35
26. Validasi kondisi dan catatan....36
27. Navigation dengan hasil bertipe....37
28. Membentuk record lokal yang valid....38
29. Praktikum form dan navigation....39
30. Inspector, breakpoint, dan log....40
31. Praktikum empat state UI....41
32. Review dan handover day-1....42
33. Mengapa layer perlu dipisahkan....43
34. Struktur folder dan dependency....44
35. Constructor injection dan ownership....45
36. Parsing JSON dengan batas jelas....46
37. Serialisasi record dan input create....47
38. Kontrak endpoint latihan....48
39. Menjalankan mock API mandiri....49
40. Base URL emulator dan device....50
41. Manifest jaringan debug dan release....51
42. Praktikum refactor dan health....52
43. Login demo dan token....53

44. Secure storage dan auth header....54
45. GET dengan status dan timeout....55
46. Membaca response daftar....56
47. POST dan hasil server....57
48. Submit guard dan tombol busy....58
49. Memetakan error ke tindakan....59
50. Timeout POST dan duplikasi....60
51. Praktikum login, GET, dan POST....61
52. ChangeNotifier sebagai pemilik state....62
53. UI mengamati state controller....63
54. Snapshot cache dan fetchedAt....64
55. Kebijakan fallback yang terbatas....65
56. Startup, logout, dan hasil lama....66
57. Praktikum session dan cache....67
58. Search, filter, dan pagination....68
59. Diagnosis koneksi dan parsing....69
60. Praktikum matriks kegagalan....70
61. Handover dan review day-2....71
62. Plugin dan konfigurasi native....72
63. Kontrak foto lokal dan ID inspeksi....73
64. Memilih galeri atau kamera....74
65. Cancel, error, dan operasi busy....75
66. Recovery lost data pada startup....76
67. MethodChannel sebagai pengayaan....77
68. Praktikum foto Android....78
69. Unit, widget, dan device test....79
70. Unit test batas validator....80
71. Unit test kontrak model....81
72. Widget test form invalid....82
73. Fake repository dan state async....83
74. Quality gate sebelum release....84
75. DevTools dan profile mode....85
76. Mengukur satu perbaikan....86
77. Gambar dan resource memory....87
78. Praktikum testing dan profiling....88
79. Polishing alur end-to-end....89
80. Batas kode bersama dan toolchain iOS....90
81. Identitas dan versi Android....91
82. Membuat keystore latihan....92
83. Properties dan konfigurasi signing....93
84. Build APK dan AAB....94
85. Verifikasi signer dan instalasi....95
86. Smoke test release dan distribusi....96
87. Troubleshooting build dan release....97
88. Praktikum release dan handover akhir....98
A1. Merakit contoh UI lokal....99
Validator input — 1/1....100
Model dan JSON — 1/2....101
Model dan JSON — 2/2....102
Form inspeksi lokal — 1/4....103
Form inspeksi lokal — 2/4....104
Form inspeksi lokal — 3/4....105
Form inspeksi lokal — 4/4....106
Aplikasi daftar dan detail — 1/5....107
Aplikasi daftar dan detail — 2/5....108

Aplikasi daftar dan detail — 3/5 109
Aplikasi daftar dan detail — 4/5 110
Aplikasi daftar dan detail — 5/5 111
Server mock Python — 1/6 112
Server mock Python — 2/6 113
Server mock Python — 3/6 114
Server mock Python — 4/6 115
Server mock Python — 5/6 116
Server mock Python — 6/6 117
A2. Signing Kotlin DSL — persiapan 118
A3. Signing Kotlin DSL — release 119
A4. Soal pemahaman hari pertama 120
A5. Soal pemahaman data dan release 121
A6. Pembahasan hari pertama 122
A7. Pembahasan data dan release 123
A8. Catatan toolchain dan checkpoint 124
A9. Checklist perilaku inti 125
A10. Log pengujian dan profil 126
A11. Template release notes 127
A12. Istilah yang dipakai dalam buku 128
Sumber materi dan rujukan resmi 129
Bukti Aplikasi pada iPhone 16 Simulator 130

Tentang buku ini

Buku ini merupakan panduan pembelajaran Flutter Android untuk program intensif tiga hari. Materi dikembangkan dari silabus dan presentasi Hari 1, Hari 2, serta Hari 3 yang dilampirkan. Identitas PT. Dok Pantai Lamongan dan PT Arunika Olah Pancarana ditampilkan melalui kedua logo asli pada materi sumber.

Pembaca diasumsikan telah mengenal pemrograman berorientasi objek dan memiliki pengalaman dasar Kotlin atau Android. Pembaca yang baru mengenal Dart dapat mempelajari bab fondasi lebih dahulu sebelum mengikuti praktikum kelas. Satu studi kasus dipakai secara konsisten: pencatatan inspeksi peralatan dengan data dummy.

Tujuan buku adalah membantu peserta menjelaskan keputusan kode, mempraktikkan perubahan, dan membuktikan hasilnya. Contoh singkat pada halaman konsep merupakan fragmen yang dipasang pada class atau method yang dibahas. Lampiran implementasi menyediakan contoh UI lokal yang lebih lengkap. Mock API mandiri disediakan sebagai listing sehingga pembelajaran tidak bergantung pada folder praktik yang tidak ikut dilampirkan.

Scope wajib berakhir pada aplikasi latihan dan build Android. Foto masih lokal, cache bersifat baca saja, dan proses iOS hanya pengenalan. Aplikasi latihan ini bukan spesifikasi proses bisnis resmi perusahaan dan belum dinyatakan sebagai aplikasi produksi.

Identitas edisi	Keterangan
Judul	Flutter Android: Dari UI hingga Release
Edisi	Materi pembelajaran • Oktober 2026
Bahasa	Bahasa Indonesia; istilah dan kode teknis asli
Format cetak	A5, margin jilid bergantian, skala 100%

PANDUAN BUKU

Cara menggunakan buku

Ikuti urutan belajar: pahami kontrak data, jalankan perubahan kecil, periksa hasil, lalu simpan checkpoint. Membaca listing tanpa menjalankan aplikasi tidak cukup untuk memastikan bahwa lifecycle, jaringan, dan konfigurasi perangkat bekerja.

Pada halaman konsep, baca penjelasan sebelum menyalin kode. Perhatikan lokasi pemasangan fragmen, dependency yang diperlukan, dan data masukan yang diharapkan. Ketika sebuah halaman menyebut controller, gunakan field milik State atau class yang dijelaskan; jangan membuat instance baru di dalam build.

Pada halaman praktikum, kerjakan langkah berurutan dan simpan bukti: screenshot, response HTTP, hasil test, atau log build. Bukti harus menunjukkan perilaku pengguna, bukan hanya bahwa file telah dibuat. Bila terhalang prasyarat, tulis penyebab, PIC, dan langkah lanjut; jangan mengisi status lulus tanpa demonstrasi.

Gunakan daftar isi untuk menuju topik, lalu tandai perubahan pada source melalui commit. Materi pengayaan dapat dipelajari setelah seluruh acceptance inti selesai. Dua orang dapat bekerja berpasangan: seorang menulis kode dan seorang memeriksa aturan serta hasil. Tukar peran setiap sesi agar keduanya memahami implementasi.

Penanda	Cara membacanya
Konsep	Alasan, perilaku, dan batas implementasi
Contoh kode	Fragmen pendidikan kecuali dinyatakan lengkap
Latihan terarah	Tugas singkat dengan hasil yang dapat diamati
Praktikum	Urutan kerja untuk mencapai checkpoint
Acceptance	Kriteria perilaku yang harus dibuktikan

PANDUAN BUKU

Peta perjalanan tiga hari

Aplikasi dibangun secara bertahap. Hari pertama membuktikan alur daftar, detail, dan form menggunakan data lokal. Pada tahap ini peserta berlatih memahami widget tree, constraint, ownership state, dan hasil navigation tanpa kompleksitas jaringan.

Hari kedua mempertahankan alur pengguna yang sama, tetapi sumber data berpindah ke REST API. Pemisahan layer membantu membedakan masalah tampilan dari request, parsing, dan kebijakan cache. Login, session, error HTTP, timeout, serta data tersimpan menjadi bagian dari perilaku aplikasi.

Hari ketiga memeriksa integrasi platform dan kualitas. Foto dipasang pada record yang tepat, test membuktikan aturan, dan profiling membantu memilih satu perbaikan berdasarkan pengukuran. Setelah alur utama stabil, peserta menyiapkan identitas, signing, serta APK dan AAB.

Setiap hari memiliki 6,5 jam efektif. Buku memuat bahan baca dan pendalaman yang lebih luas daripada waktu presentasi. Trainer memilih bagian inti sesuai jadwal silabus; latihan tambahan dikerjakan sebelum atau sesudah kelas. Memperpanjang bacaan tidak mengubah program menjadi pelatihan iOS atau offline synchronization penuh.

- Hari 1: UI/model/form
- Hari 2: API/session/cache
- Hari 3: foto/test/release

Checkpoint	Produk pembelajaran
day-1	UI lokal, model, validator, screenshot
day-2	Login, API, cache, skenario error
day-3	Foto, test, profiling, APK/AAB, handover

Kontrak studi kasus inspeksi

DPL Inspection Training mencatat hasil pengamatan sebuah aset. Kode aset dan lokasi menunjukkan apa yang diperiksa. Kondisi menggunakan dua nilai mesin: good dan needs_action. Label pengguna boleh diterjemahkan, tetapi nilai yang dikirim ke API tetap mengikuti kontrak.

Input dinormalisasi dengan trim sebelum validasi dan penyimpanan. needs_action berarti ada temuan yang perlu tindak lanjut; catatan minimal sepuluh karakter diperlukan agar temuan tidak hanya berupa kata singkat. good mengizinkan catatan kosong. Batas panjang menjaga konsistensi antara UI dan server.

ID lokal hanya digunakan pada prototipe hari pertama. Setelah POST API berhasil, gunakan ID yang diberikan server. createdAt dicatat dalam UTC dan ditampilkan dalam waktu lokal perangkat. Jangan membuat timestamp server palsu pada client untuk menutupi response yang tidak sesuai kontrak.

Seluruh record contoh adalah data dummy. Daftar inspeksi, foto, dan nama pengguna pada latihan dipakai untuk mempelajari alur teknis. Proses inspeksi operasional, otorisasi peran, persetujuan temuan, serta retensi dokumen perlu analisis tersendiri sebelum pengembangan nyata.

Field	Aturan
assetCode	Wajib, trim, maksimum 30 karakter
location	Wajib, trim, maksimum 80 karakter
condition	good atau needs_action
note	Maksimum 500; needs_action minimal 10
id	String; ID server setelah POST
createdAt	UTC; ditampilkan sebagai waktu lokal

Prasyarat dan catatan versi

Sebelum kelas, setiap peserta menjalankan aplikasi template pada emulator atau perangkat Android. Instalasi SDK dan download dependency sebaiknya selesai pada H-3 sampai H-1. Waktu kelas digunakan untuk belajar aplikasi, bukan menunggu instalasi toolchain.

Catat versi Flutter, Dart, JDK yang digunakan Flutter, Android SDK, dan dependency hasil resolver. Gunakan SDK stabil yang kompatibel dengan paket yang dipilih trainer. pubspec.lock harus disimpan bersama source agar seluruh anggota tim menggunakan resolusi dependency yang sama.

Perintah flutter pub add pada buku mengikuti resolver SDK di mesin peserta. Buku tidak memaksa nomor versi yang mungkin tidak sesuai lingkungan tersebut. Periksa README paket untuk minimum SDK, perubahan konfigurasi native, dan perilaku backup. Setelah kelas dimulai, hindari upgrade besar tanpa alasan dan tanpa checkpoint.

Contoh kode disusun untuk pembelajaran dan ditinjau terhadap dokumentasi resmi pada 6 Oktober 2026. Build Flutter dan perangkat Android tidak dijalankan di lingkungan penyusunan buku. Keberhasilan run, test, plugin, dan signing harus dibuktikan pada mesin kelas menggunakan lembar pengujian yang tersedia.

```
flutter --version
dart --version
flutter doctor -v
flutter devices
flutter doctor --android-licenses
```

Latihan terarah: Simpan hasil doctor dan versi ke README. Tunjukkan aplikasi template berjalan; beri catatan terpisah untuk warning platform yang tidak dipakai.

Ruang lingkup dan hasil akhir

Alur inti meliputi login, membaca daftar, membuka detail, membuat inspeksi, mencari dan memfilter data, menampilkan kegagalan dengan recovery, melampirkan preview foto lokal, serta logout. UI harus menjelaskan apakah data berasal dari server atau snapshot cache.

Cache bukan database transaksi offline. Saat fallback aktif, create dinonaktifkan. Tidak ada antrean POST, konflik data, ataupun upload otomatis setelah jaringan pulih. Perbedaan ini perlu disebutkan dalam demo dan release notes agar pengguna tidak mengira catatan baru dapat disimpan ketika offline.

Foto pada latihan merupakan preview lokal yang terkait dengan ID inspeksi. File dari picker dapat bersifat sementara. Persistensi attachment, penyimpanan permanen, kompresi, dan upload multipart menjadi pengembangan lanjutan. Jangan menampilkan label “terunggah” bila file belum dikirim ke server.

Kelulusan dinilai dari perilaku dan bukti. Semua acceptance hari pertama dan kedua harus diperiksa sebelum melanjutkan. Pada hari ketiga, peserta menyerahkan source, hasil test, build Android, identitas signer, smoke test, dan batas implementasi. Distribusi Google Play bersifat opsional menurut kesiapan akun; overview iOS tidak menggantikan latihan Android.

Wajib	Pengayaan / lanjutan
Dart, UI, form, navigation	Animasi kompleks
REST, session, cache read-only	Offline sync, database dan migrasi
Foto lokal, test, profiling	Persistensi dan upload foto
APK/AAB dan signer latihan	Store review dan pipeline CI/CD

HARI 1 • FONDASI

01. Flutter sebagai UI deklaratif

Flutter mendeskripsikan layar sebagai susunan widget. Alih-alih mencari sebuah view lalu mengubah teksnya satu per satu, kode membentuk tampilan berdasarkan state saat ini. Ketika state berubah, framework dapat membangun kembali bagian tree yang relevan.

Untuk pengembang Kotlin, model data dan fungsi tetap terasa familier. Perubahan utama adalah cara memikirkan tampilan. build seharusnya menjawab “apa yang terlihat dengan state ini?”, bukan “request mana yang perlu dijalankan sekarang?”. Pemanggilan build dapat terjadi berulang karena banyak alasan.

Widget adalah konfigurasi tampilan yang umumnya ringan. State yang bertahan selama hidup sebuah layar berada pada object State atau controller yang jelas pemiliknya. Memisahkan konfigurasi dan data yang berubah membantu menjelaskan mengapa sebuah tampilan ter-reset atau mengapa hasil async datang ke layar yang sudah ditutup.

Mulailah dengan MaterialApp dan satu layar daftar. Tambahkan perubahan kecil, jalankan, dan lihat dampaknya. Hindari menyalin struktur besar sebelum dapat menjelaskan widget root, widget layar, dan data yang membuatnya berubah.

```
import 'package:flutter/material.dart';

void main() => runApp(const TrainingApp());

class TrainingApp extends StatelessWidget {
  const TrainingApp({super.key});
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: Scaffold(body: Text('Inspeksi')),
    );
  }
}
```

Latihan terarah: Ubah label pada layar, lakukan hot reload, lalu jelaskan hubungan antara widget dan tampilan yang dihasilkan.

02. Membuat dan menjalankan proyek

Jalankan flutter create dari folder kerja yang dapat ditulis. Nama package Dart menggunakan huruf kecil dan underscore. Buku memakai dpl_inspection agar import pada contoh dan test konsisten. Android menjadi satu-satunya platform praktik wajib.

Setelah create selesai, buka folder proyek yang memiliki pubspec.yaml. Terminal IDE juga harus berada di folder itu. Banyak kegagalan pub get atau run berasal dari terminal yang masih berada di folder induk, bukan dari dependency yang rusak.

Pilih satu emulator atau physical device melalui flutter devices. Jika ada lebih dari satu target, gunakan opsi -d dengan ID yang ditampilkan. Perangkat nyata memerlukan USB debugging dan persetujuan koneksi dari pemilik perangkat. Pastikan app template benar-benar terlihat sebelum mengubah source.

Simpan output doctor yang relevan jika run gagal. Baca penyebab pertama: license, SDK, JDK, device, atau Gradle. Menjalankan clean berkali-kali tidak memperbaiki toolchain yang belum lengkap. Trainer membantu menyelesaikan prasyarat sebelum peserta melanjutkan ke model data.

```
flutter create --platforms=android dpl_inspection
cd dpl_inspection
flutter pub get
flutter devices
flutter run

# Bila target lebih dari satu:
flutter run -d ID_DEVICE
```

Latihan terarah: Buktikan template berjalan dan catat ID device, jenis target, serta versi SDK pada README.

03. Mengenali struktur proyek

Folder lib memuat kode Dart aplikasi. Folder test memuat test yang dapat dijalankan melalui Flutter test runner. Folder android adalah proyek native yang tetap berperan dalam permission, identitas, plugin, Gradle, dan signing.

pubspec.yaml mendeklarasikan nama, environment SDK, dependency, asset, dan version. Indentasi YAML menentukan struktur, sehingga kesalahan spasi dapat membuat konfigurasi tidak terbaca. pubspec.lock mencatat dependency hasil resolver; simpan file tersebut untuk proyek aplikasi.

Folder build berisi hasil sementara dan artefak build. Jangan mengedit file generated sebagai cara utama memperbaiki aplikasi. Kode di sana dapat dibuat ulang dan perubahan akan hilang. Perbaikan dilakukan pada source atau konfigurasi yang menjadi input proses build.

Pada hari pertama, cukup pisahkan model, aturan validasi, layar, dan widget reusable. Hari kedua menambahkan layer data tanpa mengganti kontrak pengguna. Struktur yang baik memudahkan menemukan pemilik suatu perilaku; jumlah folder yang banyak bukan ukuran kualitas arsitektur.

Lokasi	Tanggung jawab
lib/models/	Inspection dan parsing
lib/validators.dart	Aturan input yang dapat diuji
lib/screens/	Daftar, detail, form, login
lib/widgets/	Komponen tampilan reusable
android/	Manifest, Gradle, MainActivity
test/	Unit dan widget test

Latihan terarah: Temukan main.dart, manifest utama, dan pubspec.yaml. Jelaskan file mana yang berubah ketika menambah plugin.

04. Hot reload, restart, dan full run

Hot reload membantu iterasi banyak perubahan Dart dengan mempertahankan State yang sudah hidup. Misalnya, perubahan warna atau padding dapat terlihat tanpa mengulangi login atau mengetik ulang form. Keadaan yang dipertahankan ini juga menjelaskan mengapa nilai awal baru belum terlihat.

Hot restart membuat ulang eksekusi Dart dan state aplikasi. Gunakan ketika perlu menguji startup, nilai awal, atau perubahan yang tidak dapat diterapkan oleh reload. Data yang hanya berada di memory akan hilang, sedangkan data persistent dapat tetap ada.

Perubahan plugin, manifest, Gradle, atau kode native memerlukan penghentian dan run/build ulang. Reload tidak membangun ulang seluruh konfigurasi platform. MissingPluginException setelah menambah package tidak langsung berarti package tersebut salah.

Saat memeriksa bug, catat jenis restart yang dilakukan. Bug session startup perlu diuji dengan proses yang benar-benar dibuka kembali. Sebaliknya, bug tampilan dapat diperiksa cepat melalui reload. Pilih siklus kerja sesuai lapisan yang diubah dan hindari menganggap semua reset memiliki makna yang sama.

Operasi	Contoh penggunaan
Hot reload	Padding, widget dan banyak perubahan Dart
Hot restart	State awal dan setup Dart
Full run/build	Plugin, manifest, Gradle dan native
Restart aplikasi	Menguji pembacaan token/cache persistent

Latihan terarah: Tambahkan counter lokal, ubah label dengan reload, lalu restart. Catat state apa yang bertahan dan yang hilang.

05. Tipe, final, dan const

Tipe membantu menyatakan bentuk data yang diharapkan oleh fungsi dan widget. String untuk kode aset berbeda dari DateTime untuk waktu. Dengan model bertipe, kesalahan lebih mudah terlihat saat analisis daripada ketika pengguna membuka detail.

final berarti variabel hanya ditetapkan sekali. const membutuhkan nilai compile-time. Sebuah List final masih dapat dimutasi jika object List tersebut mutable; final tidak otomatis membuat semua isi object tidak dapat berubah. Untuk membatasi perubahan daftar dari luar controller, gunakan salinan atau tampilan unmodifiable.

Gunakan var atau inferensi ketika tipe sudah jelas dari nilai awal. Hindari dynamic pada jalur data utama hanya untuk menghilangkan error analyzer. dynamic menunda pemeriksaan ke runtime dan dapat menyembunyikan kontrak yang tidak konsisten.

Pada model Inspection, field final membuat sebuah record tidak berubah setelah constructor. Jika nanti diperlukan edit, buat object pengganti dengan aturan yang jelas. Hari pertama hanya menambah record sehingga pendekatan immutable ini sederhana untuk ditelusuri.

```
final assetCode = 'PMP-001';
const maxAssetLength = 30;
final createdAt = DateTime.now().toUtc();

final mutable = <String>['good'];
mutable.add('needs_action');

final protected = List<String>.unmodifiable(
  mutable,
);
```

Latihan terarah: Coba menambah isi protected dan bandingkan dengan mutable. Jelaskan mengapa final tidak sama dengan immutable.

06. Null safety dan fallback bermakna

Tanda ? pada tipe menyatakan bahwa nilai boleh null. String? note berbeda dari String note. Compiler meminta penanganan nilai nullable sebelum digunakan sebagai nilai non-null, sehingga jalur tanpa data dipikirkan secara eksplisit.

Operator ?? memberikan fallback ketika nilai null. Pilih fallback sesuai makna bisnis. Catatan opsional dapat memakai string kosong, sedangkan kode aset wajib seharusnya ditolak bila tidak ada. Mengganti semua field hilang dengan string kosong dapat mengubah response rusak menjadi record yang tampak valid.

Operator ! menyatakan keyakinan bahwa nilai tidak null. Ia bukan validasi dan dapat menyebabkan kegagalan runtime bila asumsi salah. Pada form, penggunaan currentState! lazim ketika Form pasti telah terpasang, tetapi nilai input eksternal tetap perlu diperiksa.

Gunakan alur kontrol yang jelas. Setelah memeriksa hasil picker null, hentikan operasi; null berarti pengguna membatalkan. Setelah hasil navigation null, jangan menambahkan record. Perilaku ini lebih mudah dibaca daripada memaksakan non-null di setiap pemakaian.

```
String? note;
final display = note ?? '';

void acceptResult(Inspection? result) {
  if (result == null) return;
  debugPrint(result.assetCode);
}

// Kode aset wajib tidak diberi fallback diam-diam.
```

Latihan terarah: Tulis tiga contoh null: cancel form, catatan opsional, dan token tidak tersedia. Tentukan tindakan yang berbeda untuk ketiganya.

07. Fungsi dan named parameter

Fungsi yang kecil memisahkan sebuah keputusan dari tampilan. Validator, filter, dan pemetaan label cocok dibuat sebagai fungsi murni ketika hasil hanya bergantung pada input. Fungsi seperti ini mudah dibaca dan diuji tanpa perangkat Android.

Named parameter membuat call site menjelaskan makna argumen. required menyatakan input harus diberikan. Ini berguna pada constructor Inspection karena beberapa field bertipe String; urutan positional yang panjang mudah tertukar tanpa error tipe.

Return type menjelaskan hasil fungsi. Validator mengembalikan String? karena null berarti valid, sedangkan String berisi pesan error. Fungsi yang tidak menghasilkan nilai memakai void atau Future<void> jika asinkron. Pilih tipe hasil yang mewakili kontrak, bukan sekadar membuat compiler diam.

Callback UI sebaiknya singkat: membaca aksi lalu memanggil method yang sesuai. Memasukkan semua validasi, request, parsing, dan navigation ke satu onPressed membuat penyebab kegagalan sulit ditelusuri. Mulailah memisahkan aturan sekarang agar refactor hari kedua lebih ringan.

```
String conditionLabel(String value) {
  return value == 'needs_action'
      ? 'Perlu tindak lanjut'
      : 'Baik';
}

String? requiredText(
  String? raw, {required String label}
) {
  return (raw ?? '').trim().isEmpty
      ? '$label wajib diisi' : null;
}
```

Latihan terarah: Panggil requiredText dengan input kosong dan spasi. Pastikan hasilnya sama setelah trim.

08. Model Inspection yang konsisten

Model menjadi jembatan antara data dan UI. Satu class Inspection dipakai oleh daftar, detail, form, dan layer data. Tanpa model bersama, sebuah layar mungkin mengira field bernama asset_code sementara layar lain memakai assetCode.

Gunakan constructor dengan semua field wajib yang menentukan identitas record. note disimpan sebagai String agar UI tidak terus memeriksa null. Nilai awal note kosong diberikan pada pembentukan record yang valid, bukan untuk menyembunyikan data wajib yang hilang.

createdAt bertipe DateTime agar dapat diproses sebagai waktu. Normalisasi ke UTC dilakukan pada boundary pembentukan atau parsing. UI memilih representasi lokal untuk pengguna. Simpan nilai mesin dan label tampilan secara terpisah; label bukan pengganti condition yang dikirim API.

Class berikut adalah fondasi. Parsing JSON dan serialisasi ditambahkan pada hari kedua. Jangan menduplikasi class dengan nama sama pada setiap screen. Import model dari satu lokasi, lalu jalankan analyzer setelah memindahkan file.

```
class Inspection {
  final String id, assetCode, location;
  final String condition, note;
  final DateTime createdAt;

  const Inspection({
    required this.id,
    required this.assetCode,
    required this.location,
    required this.condition,
    required this.note,
    required this.createdAt,
  });
}
```

Latihan terarah: Buat tiga record: dua good dan satu needs_action. Pastikan masing-masing mempunyai ID berbeda dan createdAt UTC.

09. Collection, map, dan where

List<Inspection> menyatakan sebuah kumpulan record. where memilih item yang memenuhi kondisi, sedangkan map mengubah setiap item menjadi bentuk lain. Keduanya menghasilkan iterable; toList dipakai ketika UI memerlukan daftar yang dapat diakses dengan index.

Misalnya, daftar temuan hanya memuat condition needs_action. Daftar kode aset merupakan hasil map dari semua record. Jangan menggunakan map untuk memilih data atau where untuk mengubah sebuah model karena makna operasi menjadi tidak jelas.

Perhatikan sumber data dan hasil filter. Menampilkan visibleItems tidak berarti items sumber boleh ditimpa setiap kali pengguna mengetik. Jika sumber ikut diganti oleh hasil pencarian, menghapus query tidak dapat mengembalikan record yang telah hilang.

Untuk latihan kecil, filter lokal dapat berjalan setiap input berubah. Ketika jumlah record membesar, ukur biaya perhitungan dan pertimbangkan pencarian server. Hari kedua akan menjelaskan bahwa filter pada satu halaman hasil API tidak menjamin seluruh data server telah dicari.

```
final needsAction = items.where(
  (x) => x.condition == 'needs_action',
).toList();

final assetCodes = items
    .map((x) => x.assetCode)
    .toList();

final totalFindings = needsAction.length;
```

Latihan terarah: Tambahkan satu record good dan pastikan jumlah temuan tetap. Hapus filter tanpa menghapus data sumber.

10. Future dan async/await

Future mewakili hasil yang belum tersedia. await menghentikan kelanjutan fungsi async sampai Future selesai, tetapi tidak memblokir seluruh UI seperti pekerjaan sinkron yang berat. Ketika hasil datang, State atau controller memutuskan apakah hasil masih relevan.

Pada hari pertama, delay simulasi membantu mengamati loading. Data tetap lokal sehingga fokus berada pada lifecycle dan state. Pada hari kedua, Future mewakili request jaringan, pembacaan storage, serta parsing yang selesai setelah aksi pengguna.

Gunakan try, catch, dan finally untuk mengelola hasil, kegagalan, serta pembersihan status busy. finally bukan tempat mengabaikan error; ia memastikan indikator operasi dihentikan. Setelah await pada State, periksa mounted sebelum mengubah tampilan atau memakai context.

Hindari menjalankan load dari build. Setiap rebuild dapat memulai Future baru sehingga request berulang, indikator berkedip, dan hasil lama menimpa hasil baru. Jalankan load awal dari lifecycle yang tepat atau aksi pengguna, lalu simpan hasil pada satu pemilik state.

```
Future<List<Inspection>> loadDemoItems() async {
  await Future<void>.delayed(
    const Duration(milliseconds: 600),
  );
  return demoItems;
}

// Di method async milik State:
final result = await loadDemoItems();
if (!mounted) return;
setState(() => items = result);
```

Latihan terarah: Buka layar lalu segera kembali saat loading. Pastikan tidak ada setState setelah dispose.

11. Praktikum model dan data lokal

Persiapan: aplikasi template telah berjalan dan folder lib/models tersedia. Buat inspection.dart lalu salin model yang dibahas. Buat data dummy di file terpisah agar dapat digunakan oleh daftar dan unit test.

Langkah 1: buat record PMP-001 di Workshop A, CRN-002 di Area Dock, dan GEN-003 di Utility. Berikan kondisi berbeda serta note yang memenuhi aturan. Gunakan ID local-1 sampai local-3 agar identitas mudah dilihat pada debugging.

Langkah 2: buat Future loadDemoItems dengan delay singkat. Panggil dari initState melalui sebuah method load. Simpan data hanya setelah mounted diperiksa. Langkah 3: tampilkan jumlah seluruh record dan jumlah needs_action di layar sederhana sebelum membuat daftar yang lebih lengkap.

Langkah 4: ubah salah satu condition dan periksa hasil filter. Bila hasil tidak berubah, periksa apakah UI membaca daftar yang benar atau salinan lama. Jangan memperbaiki angka secara manual pada label.

Bukti selesai: model bertipe, tiga ID unik, timestamp UTC, jumlah temuan benar, dan app tetap berjalan. Simpan screenshot serta commit. Reviewer menjelaskan mengapa model tidak dibuat ulang sebagai Map pada setiap layar.

Pemeriksaan	Hasil yang diharapkan
Jumlah data	Tiga record
Temuan	Sesuai condition pada data
Identitas	ID unik dan stabil
Async	Loading selesai tanpa lifecycle error
Analyzer	Tidak ada error baru dari model

12. Membaca widget tree

Widget tree menggambarkan hubungan parent dan child. MaterialApp berada di atas layar dan menyediakan theme serta navigator. Scaffold menyediakan struktur layar Material: app bar, body, dan tombol aksi utama.

Pada body, Column dapat menempatkan ringkasan di atas daftar. Daftar yang dapat di-scroll memerlukan tinggi terbatas. Expanded membantu ListView memakai sisa tinggi yang diberikan Column. Jika hubungan parent salah, pesan error layout sering muncul walaupun widget daftar terlihat sederhana.

BuildContext menunjukkan lokasi widget dalam tree. Context form dan context aplikasi tidak selalu memiliki ancestor yang sama. Saat memakai Navigator atau Theme, pahami dari lokasi mana lookup dilakukan. Simpan logika navigation dekat layar pemilik alur.

Gunakan Inspector untuk memilih komponen dan menelusuri parent. Ketika terjadi overflow, lihat constraint dan ukuran yang sebenarnya, bukan hanya menambah SizedBox secara acak. Diagram di bawah adalah struktur layar latihan, bukan seluruh tree internal framework.

- MaterialApp
- Scaffold: AppBar + Body
- Column: ringkasan + Expanded
- ListView: tile ID

Latihan terarah: Temukan Scaffold, Column, dan Expanded melalui Inspector. Jelaskan siapa yang memberi tinggi kepada ListView.

13. StatelessWidget dan StatefulWidget

StatelessWidget cocok untuk komponen yang tampilannya ditentukan oleh input constructor, seperti tile inspeksi atau label kondisi. Ketika parent memberikan input baru, framework dapat membangun konfigurasi tampilan baru.

StatefulWidget digunakan saat layar memiliki data yang berubah selama hidupnya: pilihan kondisi, controller teks, atau status loading lokal. Data tersebut disimpan pada class State, bukan pada instance widget yang menjadi konfigurasi input.

Pemilihan jenis widget bukan pernyataan bahwa StatelessWidget tidak pernah dibangun ulang. Keduanya dapat rebuild. Pertanyaan yang berguna adalah apakah komponen ini membutuhkan state yang dimilikinya sendiri atau cukup menerima nilai serta callback dari parent.

Hindari membuat setiap tile menjadi StatefulWidget hanya karena record dapat berubah. Daftar dapat memiliki state, lalu tile menerima record terbaru. Batas ini mengurangi tempat mutation dan membuat review lebih mudah. Untuk form, controller teks benar-benar memiliki lifecycle sehingga diletakkan pada State form.

```
class StatusLabel extends StatelessWidget {
  final String condition;
  const StatusLabel({
    super.key, required this.condition,
  });
  @override
  Widget build(BuildContext context) {
    return Text(conditionLabel(condition));
  }
}
```

Latihan terarah: Pisahkan label kondisi menjadi widget. Tunjukkan bahwa data berasal dari parent dan tidak disalin ke state yang tidak diperlukan.

14. Scaffold, SafeArea, dan aksi utama

Scaffold membantu membuat struktur layar yang konsisten. AppBar menunjukkan konteks halaman, body berisi data, dan FloatingActionButton menyediakan aksi tambah inspeksi. Gunakan judul yang menjelaskan tugas pengguna, bukan nama class internal.

SafeArea memberi ruang terhadap area sistem yang perlu dihindari. Ia bukan solusi umum untuk semua overflow. Konten panjang tetap membutuhkan scroll, dan daftar di dalam Column tetap membutuhkan constraint tinggi.

Aksi tambah membuka form dengan navigation. Tombol tidak langsung menambahkan record kosong ke daftar. Record baru hanya diterima setelah form selesai dengan input valid. Cancel tidak mengubah data.

Ketika jaringan diperkenalkan, tombol tambah mengikuti state aplikasi. Mode cache read-only dan operasi submit aktif memerlukan perilaku yang jelas. Label serta indikator pengguna harus menjelaskan alasan aksi belum tersedia; jangan hanya menyembunyikan tombol sehingga pengguna kehilangan konteks.

```
Scaffold(
  appBar: AppBar(
    title: const Text('Daftar inspeksi'),
  ),
  body: SafeArea(child: buildBody()),
  floatingActionButton: FloatingActionButton(
    onPressed: openForm,
    tooltip: 'Tambah inspeksi',
    child: const Icon(Icons.add),
  ),
)
```

Latihan terarah: Buka form melalui aksi utama lalu batalkan. Periksa bahwa jumlah daftar tetap.

15. Constraint dan ukuran tersedia

Layout Flutter dimulai dari batas yang diberikan parent. Child memilih ukuran dalam batas tersebut, lalu parent menempatkannya. Karena itu, sebuah widget tidak dapat dipahami hanya dari width atau height yang diminta; parent dapat membatasi permintaan itu.

Column dengan ListView tanpa batas tinggi dapat menghasilkan viewport unbounded. Scroll view ingin mengetahui ruang viewport, sedangkan Column dapat memberikan ruang vertikal tak terbatas kepada child non-flex. Expanded tepat digunakan bila Column sendiri berada dalam area tinggi yang terbatas.

Jangan meletakkan Expanded dalam SingleChildScrollView vertikal tanpa memahami constraint. Scroll view sering memberikan tinggi tak terbatas, sehingga tidak ada “sisa tinggi” terbatas yang dapat dibagikan. Form scroll dan body daftar memiliki pola yang berbeda.

Diagnosis dilakukan dari pesan error ke parent yang memberi constraint. Tambahkan batas pada tempat yang benar, kemudian uji kembali layar sempit dan landscape. Mengaktifkan shrinkWrap di semua ListView mungkin menghilangkan gejala tetapi dapat membuat layout dan performa tidak sesuai kebutuhan.

```
Column(
  children: [
    const Padding(
      padding: EdgeInsets.all(16),
      child: Text('Ringkasan inspeksi'),
    ),
    Expanded(
      child: ListView.builder(
        itemCount: items.length,
        itemBuilder: buildItem,
      ),
    ),
  ],
)
```

Latihan terarah: Hilangkan Expanded untuk mempelajari pesan error, lalu pulihkan. Catat parent yang memberikan tinggi terbatas.

16. Row, Flexible, dan teks panjang

Row menyusun child secara horizontal dan Column secara vertikal. Kode aset pendek mungkin terlihat baik di Row, tetapi lokasi atau note panjang dapat meminta lebar lebih besar daripada layar. Data realistis diperlukan saat memeriksa layout.

Flexible memberi child kesempatan menggunakan ruang yang dialokasikan tanpa selalu memaksa mengisinya. Expanded menggunakan fit ketat sehingga child mengisi alokasi. Untuk teks panjang di samping icon, Expanded sering memadai karena teks perlu membungkus pada sisa lebar.

Hindari ukuran tetap yang hanya cocok dengan satu perangkat. Lebar layar, skala teks, orientasi, dan keyboard dapat mengubah ruang tersedia. Gunakan padding dan spacing konsisten, lalu periksa apakah label masih terbaca ketika ukuran font sistem dinaikkan.

Overflow bukan hanya masalah estetika. Ia dapat menyembunyikan nilai atau tombol yang dibutuhkan pengguna. Perbaikan harus mempertahankan makna data; memotong semua note tanpa jalan menuju detail bukan pengganti layout yang benar.

```
Row(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    const Icon(Icons.place_outlined),
    const SizedBox(width: 8),
    Expanded(
      child: Text(item.location),
    ),
  ],
)
```

Latihan terarah: Gunakan lokasi lebih dari 60 karakter. Uji portrait, landscape, dan font besar tanpa overflow.

17. ListView.builder dan key record

ListView.builder membuat child ketika diperlukan oleh area scroll. Untuk daftar yang dapat bertambah, pola ini lebih sesuai daripada menulis seluruh item secara manual. itemCount berasal dari daftar yang sedang ditampilkan.

Gunakan ID record sebagai ValueKey. Key membantu framework mencocokkan identitas ketika posisi item berubah. Index merupakan posisi, bukan identitas. Jika record baru disisipkan di awal, index semua record lama berubah walaupun identitasnya tetap.

Tile menerima model dan callback membuka detail. Detail memakai record yang dipilih, bukan mengambil index yang mungkin sudah berubah setelah filter atau refresh. Pada integrasi server, sumber ID berasal dari response server.

Jangan menjanjikan bahwa key menyelesaikan semua masalah state. Pemilik photo, controller, dan request tetap harus benar. Key menjaga kecocokan widget, sementara business identity menentukan hubungan data. Kedua konsep saling melengkapi dan perlu dibuktikan dengan perpindahan antar-record.

```
ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, index) {
    final item = items[index];
    return ListTile(
      key: ValueKey(item.id),
      title: Text(item.assetCode),
      subtitle: Text(item.location),
      onTap: () => openDetail(item),
    );
  },
)
```

Latihan terarah: Sisipkan record di awal daftar. Pastikan detail record lama tetap menunjukkan ID dan kode aset yang sesuai.

18. Komponen reusable dan theme

Komponen reusable memiliki tujuan yang sempit dan input yang jelas. InspectionTile bertanggung jawab menampilkan satu record, sedangkan validator bertanggung jawab menilai aturan. Menggabungkan request API ke tile membuat komponen sulit digunakan dan diuji.

Constructor menerima data yang benar-benar diperlukan. Callback seperti onTap diberikan parent sehingga tile tidak menentukan seluruh alur navigation. Gunakan const ketika constructor dan nilai memungkinkan, tetapi jangan menambahkan const sebagai ritual tanpa memahami state.

Theme membuat warna dan typography konsisten. Ambil style dari Theme.of(context) agar komponen mengikuti kebijakan aplikasi. Hindari banyak ukuran teks berbeda tanpa hierarki; kode aset, lokasi, kondisi, dan catatan mempunyai prioritas baca yang berbeda.

Status tidak boleh hanya bergantung pada warna. Tampilkan label “Baik” atau “Perlu tindak lanjut”, lalu gunakan warna sebagai bantuan tambahan. Ini tetap jelas pada cetakan hitam putih, pengguna yang sulit membedakan warna, dan kondisi layar yang kurang ideal.

```
MaterialApp(
  theme: ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(
      seedColor: Colors.teal,
    ),
  ),
  home: const InspectionListScreen(),
)
```

Latihan terarah: Pisahkan InspectionTile dan StatusLabel. Periksa bahwa label kondisi tetap bermakna tanpa warna.

19. Form scroll dan keyboard

Form harus tetap dapat digunakan ketika keyboard terbuka. Konten panjang biasanya diletakkan di SingleChildScrollView dengan padding. Tombol simpan perlu dapat dijangkau tanpa memaksa pengguna menutup keyboard hanya untuk menemukan tindakan berikutnya.

Scaffold dapat menyesuaikan body terhadap inset keyboard. Uji perilaku nyata pada device; screenshot tanpa keyboard tidak membuktikan bahwa layout aman. Jangan menggabungkan scroll vertikal dan Expanded tanpa constraint yang sesuai.

Keyboard action next dapat membantu berpindah antar-field. Label tetap terlihat melalui InputDecoration dan pesan validator menjelaskan apa yang perlu diperbaiki. Note menggunakan input multi-baris agar temuan panjang tidak menjadi satu baris yang sulit dibaca.

Form yang gagal request tetap mempertahankan controller. Menutup halaman otomatis ketika validasi server gagal membuat pengguna kehilangan pekerjaan. Hari pertama mempelajari validasi lokal, sedangkan hari kedua menambahkan status submit dan recovery jaringan pada struktur yang sama.

```
SingleChildScrollView(
  padding: const EdgeInsets.all(16),
  child: Form(
    key: formKey,
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      children: buildFields(),
    ),
  ),
)
```

Latihan terarah: Isi note panjang, tampilkan keyboard, lalu scroll ke simpan. Ulangi pada landscape dan font besar.

20. Asset dan konfigurasi pubspec

Asset adalah file yang disertakan dalam paket aplikasi. Deklarasinya berada di bawah bagian flutter pada pubspec.yaml. Nama file dan path harus sesuai, termasuk perbedaan huruf besar-kecil yang dapat terlihat pada sistem operasi berbeda.

Deklarasi folder assets/images tidak membuat gambar secara otomatis. File harus benar-benar tersedia. Jika gambar opsional belum disediakan, jangan menjadikannya syarat agar daftar dan form dapat berjalan.

Setelah mengubah pubspec, jalankan pub get dan restart bila diperlukan. Periksa error YAML terlebih dahulu jika asset tidak ditemukan. Jangan memindahkan gambar ke build untuk memperbaiki masalah bundling.

Pilih asset yang ringan dan relevan dengan tugas. Gambar dekorasi tidak boleh membuat konten utama sulit dibaca. Foto inspeksi dari picker berbeda dari asset: asset ikut build, sedangkan foto pengguna merupakan data runtime dengan lifecycle dan kebijakan penyimpanan sendiri.

```
# Dalam pubspec.yaml
flutter:
  uses-material-design: true
  assets:
    - assets/images/

// Dalam Dart, setelah file tersedia:
Image.asset(
  'assets/images/equipment.png',
  fit: BoxFit.contain,
)
```

Latihan terarah: Tambahkan satu gambar opsional. Uji pesan asset hilang dengan mengganti path, lalu perbaiki konfigurasi.

HARI 1 • PRAKTIKUM

21. Praktikum daftar dan detail

Input: model dan tiga record dummy telah dibuat. Mulai dengan Scaffold yang memiliki ringkasan dan ListView.builder di dalam Expanded. Gunakan key berdasarkan ID untuk setiap tile.

Langkah 1: tampilkan kode aset, lokasi, dan label kondisi. Langkah 2: saat tile dipilih, buka detail yang menerima Inspection melalui constructor. Detail menampilkan ID, kode, lokasi, kondisi, note, dan waktu lokal dari createdAt.toLocal().

Langkah 3: buat satu reusable widget untuk label kondisi. Pastikan widget menerima condition dan tidak membaca global variable. Langkah 4: tambahkan data lokasi serta note panjang, lalu periksa layar sempit, landscape, dan font besar.

Langkah 5: buka record pertama dan kedua secara bergantian. Pastikan record detail sama dengan yang diketuk. Jika hasil salah, trace argument openDetail dan constructor screen; jangan sekadar mengganti teks pada detail.

Bukti selesai: screenshot daftar dan dua detail berbeda, tidak ada overflow, dan navigation kembali bekerja. Reviewer menunjuk bagian source yang menentukan identitas record, constraint daftar, serta pemetaan label kondisi.

Skenario	Acceptance
Tap record A	Detail menampilkan ID A
Tap record B	Detail menampilkan ID B
Lokasi panjang	Wrap dan tetap terbaca
Landscape	Daftar dapat di-scroll
Kembali	Data daftar tidak berubah

Latihan terarah: Simpan commit UI-list-detail sebelum menambahkan form.

22. Lifecycle State dan resource

initState berjalan ketika State baru dibuat dan cocok untuk setup awal yang tidak memerlukan dependency inherited yang berubah. build mendeskripsikan tampilan dan dapat dipanggil berulang. dispose dipakai ketika State selesai digunakan.

Resource yang dibuat oleh State harus dilepas oleh State. TextEditingController, subscription, atau listener yang dimiliki form tidak boleh terus hidup setelah halaman ditutup. Pemilik resource menjadi bagian penting dari desain, bukan hanya detail cleanup.

Setelah await, mounted menyatakan apakah State masih terpasang. Guard ini mencegah perubahan UI pada layar yang telah ditutup. Ia tidak otomatis membatalkan request jaringan; request dapat tetap berjalan sehingga hasil juga perlu relevansi session atau identitas.

Jangan membuat TextEditingController di dalam build. Setiap rebuild akan membuat instance baru dan input dapat berubah atau resource bocor. Buat sebagai field State, baca ketika diperlukan, lalu dispose. Hari kedua menerapkan prinsip ownership yang sama pada ChangeNotifier dan http.Client.

```
final assetController = TextEditingController();

@override
void dispose() {
  assetController.dispose();
  super.dispose();
}

// Setelah operasi async:
if (!mounted) return;
```

Latihan terarah: Buka dan tutup form berulang. Periksa bahwa controller berada pada field State dan semuanya dilepas.

23. Loading, success, empty, dan error

Loading berarti operasi sedang berlangsung; success berarti hasil tersedia. Empty adalah hasil valid dengan jumlah item nol. Error berarti operasi gagal dan hasil tidak dapat dinyatakan sebagai daftar kosong. Keempat kondisi perlu tampilan yang berbeda.

Pada empty, pengguna dapat diarahkan menambah inspeksi. Pada error, pengguna menerima pesan dan retry yang sesuai. Loading menahan interaksi yang akan memulai operasi identik. Success dapat menampilkan daftar, termasuk saat query filter tidak memiliki kecocokan.

Bedakan “belum ada inspeksi” dari “pencarian tidak menemukan hasil”. Keduanya kosong secara visual, tetapi tindakan berikutnya berbeda. Menghapus filter dapat memulihkan hasil pencarian tanpa request baru.

Tentukan prioritas state agar indikator tidak bertabrakan. Misalnya, loading awal memakai indikator penuh, sedangkan refresh dengan data lama dapat mempertahankan daftar dan indikator kecil. Latihan menggunakan pola sederhana lebih dahulu; yang penting pengguna tidak menerima success palsu ketika operasi gagal.

- Loading: operasi berjalan
- Success/empty: hasil valid
- Error: pesan + retry ke loading

Latihan terarah: Simulasikan hasil [], satu error, dan hasil tiga record. Pastikan masing-masing terlihat berbeda dan retry mempunyai feedback.

24. Load yang aman terhadap lifecycle

Method load menjadi satu tempat perubahan state daftar lokal. Guard isLoading mencegah operasi paralel yang sama. Sebelum menunggu hasil, aktifkan loading dan hapus pesan error lama agar pengguna melihat operasi baru sedang berjalan.

Di blok try, tunggu data lalu periksa mounted. Di catch, ubah errorMessage hanya jika State masih hidup. Di finally, hentikan loading dengan guard yang sama. Jangan menaruh await di callback setState; pekerjaan asinkron berada di luar mutation sinkron tersebut.

Contoh latihan mempertahankan items jika terjadi error. Pilih tampilan error dengan sengaja agar pengguna tidak mengira item lama merupakan hasil terbaru. Pada hari kedua, status cache dan fetchedAt menjelaskan sumber serta umur data.

Guard ini cocok untuk load tunggal. Search server yang berubah cepat memerlukan request identity agar hasil lama tidak menimpa query terbaru.

```
Future<void> load() async {
  if (isLoading) return;
  setState(() {
    isLoading = true; errorMessage = null;
  });
  try {
    final result = await loadDemoItems();
    if (!mounted) return;
    setState(() => items = result);
  } catch (_) {
    if (!mounted) return;
    setState(() => errorMessage = 'Gagal memuat');
  } finally {
    if (mounted) setState(() => isLoading = false);
  }
}
```

Latihan terarah: Tambahkan tombol retry dan tekan berulang. Satu load tidak boleh menghasilkan beberapa operasi identik.

25. Form dan validator field wajib

Form mengelompokkan field yang perlu divalidasi bersama. GlobalKey<FormState> disimpan sebagai field State agar identitas Form tidak berganti setiap rebuild. validate menjalankan validator seluruh field yang terpasang.

Kode aset dan lokasi diperiksa setelah trim. Input spasi bukan nilai yang bermakna. Selain wajib, periksa panjang maksimal dalam validator; counter maxLength membantu pengguna, tetapi business rule tetap dinyatakan pada fungsi agar dapat dipakai oleh jalur lain.

Validator mengembalikan null untuk input valid atau pesan yang menjelaskan perbaikan. Pesan “invalid” kurang membantu dibanding “Kode aset wajib diisi” atau “Maksimal 30 karakter”. Gunakan label yang konsisten dengan field.

Saat simpan ditekan, berhenti jika validate false. Jangan membuat record lalu mencoba menghapusnya ketika input ternyata salah. Urutan validasi, normalisasi, pembentukan record, dan navigation harus dapat ditelusuri jelas.

```
String? validateAsset(String? raw) {
  final value = (raw ?? '').trim();
  if (value.isEmpty) return 'Kode aset wajib diisi';
  if (value.length > 30) return 'Maksimal 30 karakter';
  return null;
}

TextFormField(
  controller: assetController,
  decoration: const InputDecoration(
    labelText: 'Kode aset',
  ),
  validator: validateAsset,
)
```

Latihan terarah: Uji kosong, spasi, 30 karakter, dan 31 karakter. Tulis hasil sebelum menambah field berikutnya.

26. Validasi kondisi dan catatan

condition hanya boleh good atau needs_action. Gunakan kontrol pilihan yang membatasi input, tetapi server tetap harus memeriksa nilai. UI yang terkontrol tidak menjamin request dari semua client valid.

Aturan note bergantung pada condition. Setelah trim, panjang note maksimal 500. Ketika needs_action dipilih, panjang minimal sepuluh. good mengizinkan string kosong. Fungsi murni menjaga aturan tidak tersebar pada beberapa tombol atau widget.

Perubahan condition dapat mengubah validitas note yang sudah diketik. Pastikan pengguna melihat pesan yang sesuai ketika memilih needs_action. Gunakan validasi ketika simpan dan, bila diperlukan, revalidate setelah perubahan pilihan untuk memberi feedback.

Pada latihan, panjang memakai String.length Dart. Untuk produk nyata yang menerima emoji atau karakter gabungan, tim perlu menetapkan apakah panjang berarti code unit atau grapheme. Kesepakatan client-server lebih penting daripada dua implementasi yang memiliki definisi berbeda.

```
String? validateNote(String condition, String raw) {
  final note = raw.trim();
  if (note.length > 500) {
    return 'Maksimal 500 karakter';
  }
  if (condition == 'needs_action' &&
      note.length < 10) {
    return 'Catatan temuan minimal 10 karakter';
  }
  return null;
}
```

Latihan terarah: Uji panjang 9, 10, 500, dan 501 setelah trim. Jelaskan mengapa spasi tepi tidak memenuhi syarat minimum.

27. Navigation dengan hasil bertipe

Navigator.push mengembalikan Future yang selesai ketika halaman berikutnya ditutup. Pada form, hasilnya dapat berupa Inspection atau null. null berarti tidak ada record yang dikembalikan, misalnya pengguna menekan kembali.

Layar daftar menjadi pemilik mutation daftar. Form hanya memvalidasi dan mengembalikan record. Dengan satu pemilik, record tidak ditambahkan sekaligus oleh form dan daftar. Ini mencegah duplikasi yang sering muncul ketika callback serta result navigation dipakai bersamaan.

Setelah await push, periksa mounted dan hasil null. Hasil valid disisipkan sekali. Jangan memakai index sebagai ID baru dan jangan membuat record lain dari hasil yang sebenarnya sudah lengkap.

Hari kedua mengganti record lokal dengan hasil POST server. Prinsipnya tetap: form menutup setelah penyimpanan berhasil dan daftar menerima hasil yang memiliki ID server. Kegagalan POST mempertahankan form; cancel tidak memicu request.

```
final created = await Navigator.of(context)
    .push<Inspection>(MaterialPageRoute(
  builder: (_) => const InspectionForm(),
));
if (!mounted || created == null) return;
setState(() => items = [created, ...items]);

// Di form, setelah valid:
Navigator.of(context).pop(record);
```

Latihan terarah: Tambahkan satu record lalu ulangi dengan cancel. Jumlah daftar hanya bertambah pada hasil valid.

28. Membentuk record lokal yang valid

Record dibentuk setelah seluruh validator lulus. Ambil nilai dari controller dan trim sekali pada boundary simpan. Gunakan UTC untuk createdAt lalu tampilkan waktu lokal di detail.

ID local berbasis timestamp cukup untuk latihan single-device, tetapi bukan strategi identitas terdistribusi. Pada tahap API, server menentukan ID. Jangan membawa ID lokal latihan sebagai primary key yang diasumsikan unik di semua perangkat.

Perhatikan alur simpan: validate, baca input, buat Inspection, lalu pop dengan hasil. Tidak ada request jaringan pada hari pertama. Tombol cancel cukup menutup form tanpa result. Controller dilepas oleh dispose ketika form selesai.

Jika record muncul dua kali, periksa tempat mutation sebelum mengubah generator ID. Duplikasi sering berasal dari dua jalur penambahan, bukan dari timestamp yang sama. Simpan breakpoint pada pop dan penerimaan hasil agar alurnya dapat dibuktikan.

```
if (!formKey.currentState!.validate()) return;
final now = DateTime.now().toUtc();
final record = Inspection(
  id: 'local-${now.microsecondsSinceEpoch}',
  assetCode: assetController.text.trim(),
  location: locationController.text.trim(),
  condition: condition,
  note: noteController.text.trim(),
  createdAt: now,
);
Navigator.of(context).pop(record);
```

Latihan terarah: Tambahkan note dengan spasi tepi. Detail harus menunjukkan nilai yang telah dinormalisasi.

HARI 1 • PRAKTIKUM

29. Praktikum form dan navigation

Input: daftar dan detail telah berjalan. Buat StatefulWidget InspectionForm dengan tiga controller teks, condition awal good, dan GlobalKey<FormState>. Letakkan semua resource pada field State dan dispose seluruh controller.

Langkah 1: buat field kode aset, lokasi, kondisi, dan note. Pasang validator wajib serta batas panjang. Langkah 2: gunakan SingleChildScrollView sehingga keyboard dan layar sempit tetap memberi akses simpan.

Langkah 3: terapkan validateNote pada field note dengan condition saat ini. Langkah 4: buat method save lokal yang membentuk Inspection dan mengembalikan hasil melalui Navigator.pop. Pada daftar, tunggu hasil push bertipe dan tambahkan hanya sekali.

Langkah 5: uji input kosong, hanya spasi, note temuan pendek, nilai valid, dan cancel. Jalankan dengan breakpoint jika jumlah daftar tidak sesuai. Buka detail record baru untuk memastikan seluruh field tersimpan sesuai input.

Bukti selesai: screenshot error wajib, screenshot error note temuan, screenshot record baru, dan hasil cancel tanpa mutation. Reviewer menjelaskan pemilik controller, validator, dan perubahan daftar. Simpan commit sebelum menambah simulasi error.

Data uji	Hasil
Kode/lokasi kosong	Form tetap terbuka dengan pesan
needs_action + cek	Ditolak
needs_action + note >=10	Diterima
good + note kosong	Diterima
Cancel	Tidak ada record baru

30. Inspector, breakpoint, dan log

Inspector membantu melihat hubungan widget serta ukuran. Gunakan untuk menemukan parent dari teks overflow atau ListView yang tidak dibatasi. Error layout perlu diperiksa melalui constraint, bukan hanya warna garis peringatan.

Breakpoint berguna pada validator, pembentukan record, Navigator.pop, dan penerimaan result. Perhatikan nilai mentah dan nilai setelah trim. Jika UI menampilkan pesan yang salah, pastikan condition yang diperiksa merupakan state pilihan terbaru.

Log mencatat langkah yang relevan: load mulai, load selesai, dan ID record yang dipilih. Hindari log seluruh payload tanpa tujuan. Pada hari kedua, token dan password tidak dimasukkan ke log. Nama action serta status HTTP biasanya cukup untuk diagnosis awal.

Debugging yang efektif menguji satu hipotesis. Misalnya, “form reset karena controller dibuat pada build” dapat diperiksa dengan lokasi constructor controller dan rebuild terkontrol. Setelah sumber masalah ditemukan, perbaiki sebab lalu ulangi skenario yang sama, bukan hanya melihat bahwa aplikasi tidak crash.

Gejala	Hipotesis awal
Form reset	Controller baru pada build
Detail salah	Argument atau ID record tidak sesuai
setState after dispose	Async tanpa guard mounted
Daftar duplikat	Mutation di dua tempat
List unbounded	Parent tidak memberi tinggi

Latihan terarah: Temukan satu bug dengan breakpoint. Simpan catatan gejala, hipotesis, bukti, dan perubahan yang menghilangkannya.

31. Praktikum empat state UI

Buat skenario data normal, hasil kosong, dan error sekali. Simulasi membantu memeriksa tampilan tanpa harus menunggu API. Pilihan skenario berada pada fungsi loader latihan, bukan hardcoded pada teks UI.

Langkah 1: saat load dimulai, tampilkan CircularProgressIndicator. Langkah 2: pada hasil kosong, tampilkan penjelasan dan aksi tambah. Langkah 3: saat loader melempar exception, tampilkan pesan error dan tombol retry. Retry memulai loading baru lalu dapat memulihkan data.

Langkah 4: tutup layar saat operasi berjalan untuk memeriksa mounted. Langkah 5: tekan retry berulang dan pastikan guard mencegah load identik bersamaan. Tidak boleh ada request yang dipicu hanya karena widget rebuild.

Uji juga form dengan keyboard, lokasi panjang, dan font besar. Perbaikan visual tidak selesai jika tombol masih tidak terjangkau. Pada daftar, bedakan data benar-benar kosong dari filter tanpa kecocokan ketika fitur search nanti ditambahkan.

Bukti selesai: empat screenshot state dan demonstrasi error ke retry ke success. Reviewer memeriksa bahwa error bukan diubah menjadi [], dan semua controller dilepas. Siapkan tag day-1 hanya setelah acceptance inti terpenuhi.

State	Bukti
Loading	Indikator saat Future belum selesai
Success	Data sesuai model
Empty	Pesan dan aksi yang tepat
Error	Pesan serta retry
Recovery	Retry kembali ke data benar

HARI 1 • CHECKPOINT

32. Review dan handover day-1

Demo dimulai dari daftar, membuka dua detail, lalu menguji form invalid dan valid. Setelah itu, demonstrasikan cancel, empty, error, dan retry. Reviewer tidak hanya menghitung layar; ia memeriksa identitas data serta hubungan aksi dengan state.

Simpan README yang menyebut versi SDK, device, dan langkah run. Sertakan catatan bahwa hari pertama memakai data lokal. Screenshot harus diberi nama yang menjelaskan skenario, misalnya day1-form-note-invalid.png.

Periksa git diff sebelum commit agar tidak memasukkan file yang tidak relevan. Jika proyek sudah memiliki repository, jangan melakukan git init ulang sebagai ritual. Tag day-1 menunjuk checkpoint yang dapat dibuka kembali untuk melanjutkan hari kedua.

Pertanyaan review: mengapa load tidak berada pada build; mengapa empty berbeda dari error; siapa pemilik daftar; siapa yang dispose controller; dan mengapa mounted diperiksa setelah await. Peserta yang dapat menunjukkan jawabannya pada source siap mengikuti refactor layer data.

```
git status
git add .
git diff --cached
git commit -m "Complete day 1 UI and form"
git tag day-1
```

Acceptance inti	Status yang dicatat
Daftar/detail benar	Lulus / Perlu perbaikan
Validasi wajib dan note	Lulus / Perlu perbaikan
Tambah sekali, cancel aman	Lulus / Perlu perbaikan
Empat state dan retry	Lulus / Perlu perbaikan
Layar sempit dan lifecycle	Lulus / Perlu perbaikan

33. Mengapa layer perlu dipisahkan

Pada prototipe, daftar lokal dapat dikelola oleh satu layar. Ketika API, token, cache, dan error bertambah, mencampur seluruh pekerjaan pada State membuat perubahan sulit diuji. Pemisahan layer membantu menentukan tempat suatu keputusan dibuat.

UI bertanggung jawab pada input, tampilan, dan navigation. Controller mengelola status operasi yang dibaca UI. Repository memilih sumber data serta kebijakan cache. Service menjalankan HTTP dan menerjemahkan response menjadi model atau exception. Model menyatakan kontrak data.

Pola ini adalah pendekatan sederhana untuk pelatihan, bukan keharusan memakai semua lapisan clean architecture. Nama controller pada buku mempunyai peran pengelola state tampilan, mirip view model pada pendekatan lain. Tambahkan abstraksi jika ada manfaat nyata untuk pengujian dan maintenance.

Refactor bertahap: model dan validator, lalu service, repository, serta controller. Setelah setiap perubahan, periksa analyzer dan alur day-1.

- UI: input/tampilan
- Controller: state
- Repository: sumber/cache
- Service: HTTP/parsing
- API: kontrak server

Latihan terarah: Ambil method save pada hari pertama. Tandai bagian yang kelak menjadi UI, controller, repository, dan service.

34. Struktur folder dan dependency

Folder mencerminkan tanggung jawab, tetapi pemisahan nyata terlihat pada dependency. UI tidak seharusnya membuat request HTTP langsung pada setiap build. Repository tidak membutuhkan BuildContext untuk menentukan cache fallback.

Buat service dan repository melalui constructor. Dengan cara ini, test dapat memberikan fake tanpa menjalankan server. Hindari akses global tersembunyi yang membuat satu class membaca konfigurasi dan resource dari banyak tempat.

Pemilik session membuat resource yang digunakan selama session aktif. Pemilik layar dapat membuat controller yang hidup selama layar itu aktif. Ketika logout, controller lama tidak boleh memperbarui UI session berikutnya. Siklus hidup dependency sama pentingnya dengan struktur folder.

Mulai dengan susunan berikut dan import file yang diperlukan. Tidak perlu memecah setiap baris menjadi file terpisah. Tujuan struktur adalah memudahkan tim menemukan parsing, kebijakan data, state, serta komponen tampilan secara cepat.

```
lib/
  models/inspection.dart
  services/api_service.dart
  repositories/inspection_repository.dart
  controllers/inspection_controller.dart
  storage/session_store.dart
  storage/inspection_cache.dart
  screens/login_screen.dart
  screens/inspection_list_screen.dart
  screens/inspection_form.dart
  widgets/inspection_tile.dart
```

Latihan terarah: Pindahkan model dan validator, perbaiki import, lalu jalankan analyzer sebelum menambah networking.

35. Constructor injection dan ownership

Constructor injection menyatakan kebutuhan sebuah object secara eksplisit. InspectionRepository memerlukan ApiService; controller memerlukan repository. Saat membaca constructor, reviewer dapat mengetahui dependency tanpa menelusuri global variable.

Object yang menerima dependency belum tentu menjadi pemilik disposal dependency tersebut. Jika http.Client dibagikan untuk satu session, layar detail tidak boleh menutupnya ketika kembali. Tentukan ownership pada tempat object dibuat dan dokumentasikan kapan ia dilepas.

Test menjadi lebih terkendali bila controller bergantung pada kontrak repository. Fake dapat mengembalikan success, empty, atau error tanpa akses jaringan. Hal ini membantu menguji transisi state dan guard submit dengan hasil yang dapat diulang.

Untuk aplikasi latihan, satu session scope sudah cukup. Hindari membuat service, repository, dan controller baru pada setiap rebuild. Buat sekali melalui lifecycle atau composition root, lalu serahkan input ke widget. Ini menjaga state dan resource tetap konsisten.

```
class InspectionRepository {
  final ApiService api;
  InspectionRepository(this.api);
}

class InspectionController extends ChangeNotifier {
  final InspectionRepository repository;
  InspectionController(this.repository);
}

// Pemilik session merangkai dependency sekali.
```

Latihan terarah: Gambar daftar pemilik object: client, service, repository, dan controller. Tuliskan siapa yang menutup masing-masing resource.

36. Parsing JSON dengan batas jelas

JSON hasil decode belum menjadi model yang valid. Field wajib perlu diperiksa tipenya; condition harus salah satu nilai kontrak dan timestamp harus dapat diparse. Parsing gagal harus menjadi error kontrak, bukan daftar kosong.

Slide memakai cast sederhana untuk mengenalkan fromJson. Untuk panduan buku, parsing pada lampiran diperkuat dengan FormatException dan pemeriksaan eksplisit. Akibatnya, unit test data hilang mengharapkan FormatException, bukan TypeError dari cast. Ini merupakan perubahan yang disengaja agar error lebih bermakna.

note yang tidak tersedia dapat menjadi string kosong karena opsional menurut kontrak. Field wajib seperti id, assetCode, dan location tidak mendapat fallback palsu. createdAt pada response server harus membawa informasi zona waktu; contoh server menghasilkan UTC dengan akhiran Z.

Pisahkan parsing response daftar dari parsing satu record. Body GET harus mempunyai items berupa List dan total berupa angka yang benar. POST 201 berisi satu object Inspection. Shape response yang berbeda memerlukan jalur parsing berbeda.

```
String readRequired(
  Map<String, dynamic> j, String key,
) {
  final v = j[key];
  if (v is! String || v.trim().isEmpty) {
    throw FormatException('Field $key invalid');
  }
  return v;
}
// Helper ini dipakai pada parsing field wajib.
// Model lengkap tersedia pada lampiran.
```

Latihan terarah: Uji payload tanpa id, condition asing, dan timestamp rusak. Ketiganya harus gagal parsing secara jelas.

37. Serialisasi record dan input create

toJson dipakai untuk snapshot record yang sudah lengkap. Body create berbeda: client mengirim kode aset, lokasi, kondisi, dan note; ID serta waktu pembuatan ditentukan server. Jangan mengirim seluruh object lokal hanya karena tersedia method toJson.

Pemisahan DTO input dapat dipakai pada proyek lebih besar. Untuk latihan, Map<String, dynamic> dengan empat field cukup selama kontraknya jelas. Normalisasi dan validasi dilakukan sebelum request, lalu server memeriksa lagi.

Saat POST berhasil, parse response menjadi Inspection. Gunakan nilai dari server karena server dapat memberikan ID, timestamp, atau normalisasi. Menampilkan record lokal sebagai hasil final sebelum response dapat mengaburkan kegagalan simpan.

Cache memakai toJson dengan UTC ISO 8601. Snapshot schema rusak dihapus; versioning serta migrasi menjadi tindak lanjut.

```
Map<String, dynamic> toJson() => {
  'id': id,
  'assetCode': assetCode,
  'location': location,
  'condition': condition,
  'note': note,
  'createdAt': createdAt.toUtc().toIso8601String(),
};

final createBody = {
  'assetCode': assetController.text.trim(),
  'location': locationController.text.trim(),
  'condition': condition,
  'note': noteController.text.trim(),
};
```

Latihan terarah: Bandingkan body create dengan toJson. Jelaskan mengapa ID client tidak dikirim ke POST latihan.

38. Kontrak endpoint latihan

Kontrak API disepakati sebelum wiring UI. GET /health dapat dipakai tanpa token untuk memeriksa apakah server hidup. POST /auth/login menerima kredensial dummy dan mengembalikan token serta user.

GET /inspections membutuhkan Bearer token dan menghasilkan object items serta total. POST /inspections menerima input create dan mengembalikan satu record baru dengan status 201. Body error menyediakan message yang dapat ditampilkan sesuai kebijakan aplikasi.

Mock server pada lampiran dibuat untuk buku ini agar dapat dijalankan mandiri dengan Python standard library. Ia bukan backend produksi. Record hidup di memory dan hilang ketika proses server dimulai ulang. Token demo tetap dan tidak memiliki refresh atau expiry nyata.

Sebelum menghubungkan Flutter, uji login benar/salah, GET tanpa token, POST invalid, dan POST valid melalui REST client. Jika kontrak belum terbukti pada REST client, debugging UI dan network sekaligus akan membuat diagnosis lebih sulit.

Endpoint	Input / output
GET /health	200: status ok
POST /auth/login	200 token/user; 401 kredensial salah
GET /inspections	200 items/total; Bearer wajib
POST /inspections	201 record; 422 input invalid
Flags mock	delay, fail-list, fail-create

Latihan terarah: Catat status HTTP dan shape body setiap endpoint. Gunakan record dummy agar hasil mudah dibandingkan.

39. Menjalankan mock API mandiri

Simpan listing Python pada lampiran sebagai praktik/mock_api/mock_api.py. Jalankan dari root proyek atau ubah path sesuai lokasi file. Python 3 diperlukan; server menggunakan standard library sehingga tidak membutuhkan pip tambahan.

Server bind pada 0.0.0.0 agar dapat diakses oleh emulator dan perangkat di LAN. Gunakan jaringan latihan yang dikelola trainer. Kredensial trainer/training123 hanya untuk demo. Jika port 8000 telah digunakan proses lain, pilih port berbeda lalu ubah semua base URL terkait.

Uji health dari komputer dengan http://127.0.0.1:8000/health. Setelah itu, uji dari target Android melalui alamat yang benar. Keberhasilan browser komputer tidak otomatis membuktikan route dari device.

Flag --delay menunda response untuk menguji timeout. --fail-list dan --fail-create menghasilkan kegagalan server pada endpoint terkait. Restart server mengembalikan data awal, sehingga catat setiap restart saat membandingkan hasil create. Gunakan beberapa terminal bila perlu: satu untuk server, satu untuk Flutter.

```
python praktik/mock_api/mock_api.py --port 8000

# Simulasi timeout (client menunggu 10 detik):
python praktik/mock_api/mock_api.py --delay 11

# Simulasi 500 untuk daftar:
python praktik/mock_api/mock_api.py --fail-list

# Dependency Flutter:
flutter pub add http flutter_secure_storage
flutter pub add shared_preferences
```

Latihan terarah: Jalankan health 200 dan login demo. Simpan response yang sudah dibersihkan dari credential/token untuk bukti.

40. Base URL emulator dan device

127.0.0.1 pada perangkat Android menunjuk perangkat Android itu sendiri, bukan komputer yang menjalankan mock. Android Emulator bawaan memiliki alamat khusus 10.0.2.2 untuk mencapai host emulator. Perangkat fisik menggunakan IP LAN komputer yang dapat dirutekan.

Pada physical device, komputer dan perangkat perlu berada pada jaringan yang saling terhubung. Periksa firewall, port server, serta isolasi Wi-Fi. Mengubah kode parsing tidak memperbaiki connection refused atau route yang belum ada.

Simpan base URL dalam satu konfigurasi melalui dart-define. Nilai ini menentukan endpoint, bukan tempat menyimpan secret. Kode dan nilai konfigurasi pada aplikasi dapat diperiksa; credential tetap harus diberikan melalui mekanisme session yang tepat.

Gunakan host HTTPS staging yang nyata untuk release smoke test. HOST_STAGING pada buku adalah placeholder yang harus diganti oleh PIC. Bila staging belum tersedia, tandai pengujian API release terhalang prasyarat, sementara build serta signature tetap dapat diverifikasi.

```
const apiBaseUrl = String.fromEnvironment(
  'API_BASE_URL',
  defaultValue: 'http://10.0.2.2:8000',
);

// Terminal, satu baris:
// flutter run --dart-define=API_BASE_URL=http://10.0.2.2:8000
```

Target	Alamat mock
Android Emulator	http://10.0.2.2:8000
Physical Android	http://IP_LAN_KOMPUTER:8000
Komputer host	http://127.0.0.1:8000
Release	HTTPS staging milik PIC

41. Manifest jaringan debug dan release

Permission INTERNET harus tersedia pada manifest utama agar build release dapat menggunakan jaringan. Keberhasilan debug tidak selalu membuktikan bahwa permission release telah benar, karena konfigurasi debug dapat berbeda.

HTTP mock lokal diizinkan khusus konfigurasi debug. Tambahkan usesCleartextTraffic pada application debug atau konfigurasi jaringan debug yang sesuai proyek. Pertahankan struktur XML dan namespace Android; fragmen di bawah ditempatkan pada file yang benar, bukan ditempel sebagai satu file gabungan.

Untuk release, gunakan HTTPS staging dengan sertifikat yang valid. Jangan menonaktifkan validasi sertifikat untuk menghilangkan error TLS. Periksa host, chain certificate, dan waktu perangkat. Konfigurasi jaringan merupakan bagian dari penerimaan release.

Setelah mengubah manifest, lakukan full run agar perubahan native diterapkan. Uji lagi dari target Android. Catat apakah error merupakan cleartext blocked, connection refused, timeout, atau TLS; tiap gejala menunjuk lapisan diagnosis yang berbeda.

```
<!-- android/app/src/main/AndroidManifest.xml -->
<!-- Di dalam manifest, sebelum application: -->
<uses-permission
  android:name="android.permission.INTERNET" />

<!-- android/app/src/debug/AndroidManifest.xml -->
<!-- Tambahkan atribut pada application debug: -->
<application android:usesCleartextTraffic="true" />
```

Latihan terarah: Periksa manifest merged dari Android Studio. Pastikan INTERNET terdapat pada varian release dan HTTP mock hanya dibolehkan untuk debug.

HARI 2 • PRAKTIKUM

42. Praktikum refactor dan health

Mulai dari checkpoint day-1 yang telah lulus. Jangan mengubah aturan form. Buat folder model, services, repositories, controllers, dan storage. Pindahkan model serta validator lalu perbaiki import.

Langkah 1: tambahkan package HTTP dan storage melalui pub add. Langkah 2: simpan serta jalankan mock API dari lampiran. Uji health dari REST client komputer, kemudian pastikan alamat target Android benar.

Langkah 3: buat ApiService yang menerima base URL dan http.Client melalui constructor. Tambahkan health method dengan timeout. Langkah 4: buat repository dan controller dengan constructor injection; untuk tahap ini cukup buktikan wiring dependency dan response health.

Langkah 5: periksa manifest utama serta debug, kemudian lakukan full run. Bila gagal, tentukan lapisannya: proses server, route, firewall, permission, TLS/HTTP, status, atau parsing. Jangan mengubah UI untuk menyembunyikan kegagalan.

Bukti selesai: struktur source yang jelas, response health 200, base URL sesuai target, dan alur day-1 tetap bekerja. Simpan commit refactor-health agar integrasi login dapat dimulai dari kondisi yang diketahui.

Pemeriksaan	Bukti
Refactor	Import valid dan analyzer bersih
Server	Terminal menerima health
Target	Health dapat dicapai Android
Dependency	Dibuat melalui constructor
Regresi	Daftar/detail/form day-1 tetap benar

43. Login demo dan token

Login mengirim username serta password dummy ke POST /auth/login. Jika berhasil, aplikasi menerima token dan user. Simpan token setelah memeriksa bahwa response sesuai kontrak; jangan membuka daftar hanya karena status 200 tanpa token valid.

Password tidak disimpan sebagai kebutuhan session. Token digunakan untuk request berikutnya melalui Authorization Bearer. Token demo pada mock adalah nilai tetap untuk pembelajaran; ia tidak menjelaskan seluruh keamanan autentikasi produksi.

LoginScreen mempunyai controller input dan status busy. Tombol dinonaktifkan selama login agar klik berulang tidak membuat operasi identik. Login salah mempertahankan layar dan memberi pesan. Setelah await, mounted atau session generation diperiksa sebelum navigation.

Hindari menaruh credential demo dalam aplikasi produksi. Pada kelas, nilainya dipakai agar seluruh peserta dapat membuktikan alur yang sama. Release notes harus menyebut backend latihan dan tidak mengklaim session telah menerapkan refresh token, role permission, atau revocation produksi.

```
POST /auth/login
Content-Type: application/json

{"username":"trainer","password":"training123"}

HTTP 200
{"token":"training-demo-token",
 "user":{"id":"u1","name":"Trainer"}}
```

Latihan terarah: Uji password benar, salah, dan klik login berulang. Token hanya disimpan pada response sukses yang valid.

44. Secure storage dan auth header

Token merupakan data session dan disimpan menggunakan secure storage sesuai paket yang dipilih. Shared preferences pada latihan dipakai untuk snapshot, bukan token. Baca konfigurasi Android dan backup pada README versi paket yang terpasang.

Sebelum request terlindungi, baca token. Jika tidak ada atau kosong, arahkan alur ke login; jangan mengirim header Bearer null. Bentuk header pada satu tempat agar format konsisten di GET dan POST.

Secure storage membantu penyimpanan pada perangkat, tetapi tidak membuat seluruh sistem otomatis aman. Session tetap membutuhkan validasi server, HTTPS pada release, serta pembersihan saat logout atau 401. Token yang tersimpan bukan bukti bahwa server masih menerima session.

Storage bersifat async. Saat logout, hasil session lama diabaikan melalui ownership dan generation. Token tidak dicetak pada log.

```
const secure = FlutterSecureStorage();
await secure.write(
  key: 'session_token', value: token,
);

Future<Map<String, String>> authHeaders() async {
  final t = await secure.read(key: 'session_token');
  if (t == null || t.isEmpty) {
    throw StateError('Login diperlukan');
  }
  return {
    'Authorization': 'Bearer $t',
    'Content-Type': 'application/json',
  };
}
```

Latihan terarah: Restart aplikasi dan pastikan token dibaca, lalu periksa session dengan request server sebelum menampilkan data.

45. GET dengan status dan timeout

Service mengirim GET dan memeriksa status sebelum parsing success body. Response 401 berbeda dari 500; keduanya tidak boleh dimasukkan ke parser items seolah response 200. Exception bertipe membantu repository membuat keputusan yang tepat.

Timeout sepuluh detik dipakai untuk membatasi waktu tunggu latihan. Nilainya bukan standar universal. Pilih berdasarkan pengalaman pengguna dan karakter server. Future.timeout dapat menghentikan penantian client tanpa menjamin request jaringan di server berhenti.

Perhatikan body dan status secara terpisah. 200 dengan JSON rusak adalah kegagalan kontrak; 200 dengan items [] merupakan empty yang valid. Perbedaan ini harus tetap terlihat pada controller dan UI.

http.Client dibuat oleh pemilik dependency dan ditutup ketika scope selesai. Method GET tidak seharusnya menutup client bersama setelah setiap request. Catat route, status, dan durasi yang relevan; token di header tidak dicetak untuk debugging.

```
final response = await client.get(
  Uri.parse('$baseUrl/inspections?per_page=50'),
  headers: await authHeaders(),
).timeout(const Duration(seconds: 10));

if (response.statusCode == 401) {
  throw ApiException(401, 'Session berakhir');
}
if (response.statusCode != 200) {
  throw ApiException(
    response.statusCode, 'Daftar gagal dimuat',
  );
}
```

Latihan terarah: Uji 200, 401, dan delay 11 detik. Tunjukkan bahwa setiap hasil menghasilkan tindakan berbeda.

46. Membaca response daftar

Body GET berbentuk object dengan items dan total. jsonDecode dapat menghasilkan berbagai jenis nilai sehingga pemeriksaan shape dilakukan sebelum iterasi. Setelah items dipastikan List, setiap element harus berupa object yang cocok dengan model.

total menggambarkan jumlah di server menurut kontrak. Jumlah items pada satu page dapat lebih kecil daripada total. Untuk latihan awal, request per_page=50 cukup bagi data dummy. Jangan menyatakan seluruh data sudah dimuat jika server mengatakan masih ada halaman lain.

Parsing berhenti jika ditemukan record yang tidak valid. Kebijakan latihan menampilkan error kontrak, bukan diam-diam menghapus record rusak. Mengabaikan item tanpa catatan dapat membuat jumlah UI tidak sesuai dengan data server dan menyulitkan audit.

Unit test dapat membuat body valid, items bukan List, element bukan object, dan timestamp salah. Pada halaman lampiran model terdapat validasi record yang lebih lengkap. Pisahkan assertion parsing dari assertion tampilan agar penyebab kegagalan mudah diketahui.

```
final decoded = jsonDecode(response.body);
if (decoded is! Map<String, dynamic> ||
    decoded['items'] is! List) {
  throw const FormatException('Response daftar invalid');
}
final items = (decoded['items'] as List).map((v) {
  if (v is! Map<String, dynamic>) {
    throw const FormatException('Record invalid');
  }
  return Inspection.fromJson(v);
}).toList();
```

Latihan terarah: Ganti items menjadi string pada fake response. Aplikasi harus menampilkan error kontrak, bukan empty.

47. POST dan hasil server

POST create mengirim empat field yang sudah dinormalisasi. Server memvalidasi lagi dan memberikan record lengkap pada status 201. Service parse response tersebut lalu mengembalikan Inspection ke repository atau controller.

Form ditutup setelah create berhasil. Input tetap berada pada controller teks ketika status 422, 500, atau timeout. Pesan error menjelaskan tindakan berikutnya; tidak semua error cocok diberi tombol “kirim ulang” secara otomatis.

Setelah POST sukses, daftar dapat disisipkan dengan record hasil server. Jika refresh GET berikutnya gagal, jangan mengubah create yang sudah berhasil menjadi pesan “simpan gagal”. Bedakan operasi create dan refresh supaya pengguna tidak membuat record duplikat.

Deduplicate berdasarkan ID bila menggabungkan hasil create dengan refresh atau pagination. Nilai ID server menjadi kunci hubungan detail dan foto. Pada data produksi, server mungkin memiliki kontrak idempotency; latihan tidak menganggap kontrak tersebut tersedia.

```
final response = await client.post(
  Uri.parse('$baseUrl/inspections'),
  headers: await authHeaders(),
  body: jsonEncode(fields),
).timeout(const Duration(seconds: 10));

if (response.statusCode != 201) {
  throw ApiException(
    response.statusCode, 'Penyimpanan gagal',
  );
}
return Inspection.fromJson(
  jsonDecode(response.body) as Map<String, dynamic>,
);
```

Latihan terarah: Create satu record lalu periksa ID server. Matikan server sebelum refresh dan pastikan pesan tidak mengklaim create sebelumnya gagal.

48. Submit guard dan tombol busy

Klik ganda dapat terjadi ketika pengguna tidak melihat feedback atau perangkat lambat. Guard di controller mencegah create identik berjalan bersamaan. UI juga menonaktifkan tombol dan memberi indikator agar pengguna tahu operasi sedang diproses.

Guard controller tetap diperlukan walaupun tombol disabled. Operasi dapat dipanggil oleh jalur lain atau callback yang sudah terjadwal. Kondisi isCached juga diperiksa pada controller supaya mode read-only tidak hanya menjadi batas visual.

finally mengakhiri status submit, tetapi notifikasi tidak dikirim setelah controller disposed. Setelah repository selesai, periksa apakah session atau controller masih valid sebelum menyisipkan hasil. Jika session lama telah berakhir, hasilnya tidak boleh masuk ke daftar pengguna baru.

Form memisahkan error validasi lokal dari error API. Validator mencegah input invalid sebelum request. Server error tetap mempertahankan input. Guard tidak membuktikan idempotency lintas restart atau retry timeout; itu memerlukan kontrak dengan server.

```
if (_disposed || isSubmitting || isCached) {
  return null;
}
isSubmitting = true;
emit();
try {
  final item = await repository.create(fields);
  if (_disposed) return null;
  return item;
} finally {
  isSubmitting = false;
  emit();
}
```

Latihan terarah: Tekan simpan berulang ketika mock delay aktif. Pastikan hanya satu request create berjalan dan tombol menunjukkan busy.

49. Memetakan error ke tindakan

Error yang berguna memberi makna dan recovery. 401 menyatakan session tidak diterima server. 422 berarti input tidak memenuhi aturan. 500 menunjukkan kegagalan server. Timeout berarti client tidak menerima hasil dalam batas waktu.

Pada 401, bersihkan token serta cache session lalu tampilkan login. Cache tidak digunakan untuk menutupi session yang ditolak. Pada 422, form tetap terbuka dan input dipertahankan. Tampilkan pesan field bila server menyediakan mapping yang jelas.

500 dan JSON invalid tidak memicu fallback cache menurut kebijakan latihan. Menampilkan cache untuk semua error dapat menyembunyikan bug server atau kontrak yang rusak. Fallback dipilih terbatas untuk gangguan koneksi atau timeout GET dengan snapshot valid.

Pesan teknis lengkap dapat dicatat secara aman untuk developer, sedangkan UI memakai teks yang dapat ditindaklanjuti pengguna. Jangan menampilkan stack trace sebagai label form. Simpan status dan skenario pada matriks uji agar recovery tidak hanya diasumsikan.

Kejadian	Perilaku latihan
401	Clear token/cache; login
422	Input tetap; pesan validasi
500	Error server; retry pengguna
GET timeout/network	Cache valid atau error
POST timeout	Hasil belum pasti; cek daftar
JSON invalid	Error kontrak; tanpa fallback

Latihan terarah: Tulis pesan pengguna untuk setiap baris. Pastikan pesan tidak menjanjikan bahwa POST timeout pasti belum tersimpan.

50. Timeout POST dan duplikasi

Ketika timeout terjadi, client berhenti menunggu response. Server mungkin belum menerima request, masih memproses, atau sudah menyimpan record tetapi response belum sampai. Karena itu timeout POST tidak sama dengan transaksi gagal.

Retry otomatis dapat membuat record kedua. Menonaktifkan tombol selama request mencegah klik paralel, tetapi tidak menyelesaikan retry setelah timeout. Untuk produksi, idempotency key atau identitas operasi server perlu kontrak yang disepakati dan disimpan dengan lifecycle yang tepat.

Pada latihan, tampilkan pesan bahwa hasil belum pasti dan minta pengguna memeriksa daftar sebelum mengirim ulang. Mock dengan delay memproses request lalu menunda response, sehingga kasus record tersimpan tetapi client timeout dapat didemonstrasikan.

Saat mengecek hasil, kode aset saja tidak selalu cukup sebagai identitas transaksi karena satu aset dapat diperiksa berkali-kali. Catatan waktu dan data input membantu investigasi, tetapi jaminan deduplication harus datang dari desain server. Catat batas ini pada release notes.

Client tahu	Client belum tahu
Waktu tunggu habis	Apakah server menerima request
Response belum diterima	Apakah record sudah tersimpan
UI perlu feedback	Apakah retry akan duplikat

Latihan terarah: Jalankan --delay 11, lakukan POST, lalu GET dari REST client. Bandingkan pesan Flutter dengan record yang benar-benar ada.

HARI 2 • PRAKTIKUM

51. Praktikum login, GET, dan POST

Input: health target telah lulus dan layer dasar tersedia. Buat LoginScreen dengan status busy serta controller teks. Implementasikan login service, periksa token, kemudian simpan melalui session store.

Langkah 1: login salah harus tetap pada form. Langkah 2: login benar membuka daftar dan menjalankan GET melalui controller. Parsing menghasilkan model bertipe yang dipakai tile dan detail yang sudah ada.

Langkah 3: ubah save lokal menjadi repository create. Tetap gunakan validator yang sama. Form menutup dengan record server hanya setelah response 201 valid. Daftar menyisipkan atau memperbarui record berdasarkan ID sekali.

Langkah 4: uji 422 melalui REST client dengan note pendek, kemudian pastikan error API tidak menghapus input form. Langkah 5: aktifkan delay dan tekan simpan berulang untuk membuktikan submit guard.

Bukti selesai: login 200, daftar GET 200, create 201 dengan ID server, login salah tertangani, dan satu create tidak duplikat. Screenshot/log yang dibagikan tidak memuat token. Simpan commit sebelum mengembangkan cache agar diagnosis lebih terarah.

Skenario	Acceptance
Login salah	Form tetap dan pesan jelas
Login benar	Token valid tersimpan
GET	Data server terbaca
POST valid	ID server tampil sekali
Submit ganda	Satu operasi berjalan
Error create	Input tetap tersedia

52. ChangeNotifier sebagai pemilik state

ChangeNotifier menyediakan notifikasi bagi listener. Controller menyimpan items, status loading, status submit, error, serta metadata cache. UI membaca nilai tersebut dan memanggil method ketika pengguna melakukan aksi.

notifyListeners mengabarkan bahwa state berubah; ia tidak mengetahui property mana yang berubah. Karena itu batas widget listener perlu dipilih agar rebuild tidak terlalu luas. Mulai sederhana, kemudian ukur bila rebuild menjadi masalah.

State lokal form tetap dapat berada pada State form. Tidak semua TextEditingController perlu dipindahkan ke controller aplikasi. Pisahkan data daftar yang dipakai beberapa aksi dari input sementara yang hilang ketika form dibatalkan.

Pastikan controller dibuat sekali dan dibuang oleh pemiliknya. disposed guard mencegah notifikasi setelah selesai. Pemilik session menutup controller lama saat logout, sedangkan http.Client ditutup sesuai scope yang membuatnya. Object yang hanya menerima dependency tidak otomatis bertanggung jawab menutup semua dependency.

```
bool _disposed = false;

void emit() {
  if (!_disposed) notifyListeners();
}

@override
void dispose() {
  _disposed = true;
  super.dispose();
}
```

Latihan terarah: Pindahkan daftar dari State layar ke controller. Pastikan UI mengamati perubahan tanpa duplicate list yang berbeda pemiliknya.

53. UI mengamati state controller

ListenableBuilder membangun bagian UI ketika listenable berubah. Builder membaca controller dan memilih tampilan loading, error, empty, atau data. Request tetap dipanggil dari lifecycle atau aksi pengguna, bukan dari builder.

Prioritas tampilan harus jelas. Error kontrak tidak berubah menjadi empty, cache memiliki label dan waktu, serta busy submit tidak menghilangkan input. Jika UI tidak berubah, trace mutation, emit, listener, dan nilai yang dibaca builder.

Controller tidak memakai BuildContext untuk navigation. Ia dapat memberi status sessionExpired yang ditangani pemilik UI session. Pemisahan ini memudahkan unit test controller dan mencegah context lama dipakai setelah logout.

Saat controller dibagikan, widget hanya melepas listener miliknya; pemilik controller yang melakukan dispose. Jika layar membuat sendiri controller, layar bertanggung jawab pada disposal. Perbedaan ownership perlu terlihat pada code review agar tidak ada double dispose atau resource yang tidak pernah dilepas.

```
ListenableBuilder(
  listenable: controller,
  builder: (context, _) {
    if (controller.isLoading) {
      return const Center(
        child: CircularProgressIndicator(),
      );
    }
    if (controller.errorMessage != null) {
      return buildError(controller.errorMessage!);
    }
    return buildResult(controller);
  },
)
```

Latihan terarah: Ubah fake repository dari success ke empty dan error. Pastikan builder memilih state yang tepat tanpa memulai load baru.

54. Snapshot cache dan fetchedAt

Cache latihan menyimpan snapshot daftar hasil GET sukses. fetchedAt adalah waktu client memperoleh snapshot, bukan waktu masing-masing inspeksi dibuat dan bukan jaminan data masih berlaku. Tampilkan label serta timestamp saat snapshot dipakai.

SharedPreferencesAsync dipakai untuk data sederhana berupa JSON string. Ia bukan pengganti database transaksi atau penyimpanan file foto. Snapshot kecil sesuai kebutuhan latihan; daftar besar memerlukan evaluasi storage yang lebih tepat.

Serialisasi memakai toJson pada setiap Inspection. Saat membaca, parse object cache, timestamp, dan seluruh items. Cache rusak dihapus atau diabaikan tanpa crash. Jangan mengganti fetchedAt dengan waktu saat cache dibaca karena itu membuat data lama terlihat baru.

Storage preferences tidak cocok untuk data kritis yang harus dijamin tersimpan sebagai transaksi. Untuk produksi, tentukan klasifikasi data, batas umur, per-user partition, schema version, serta invalidasi. Pada latihan, snapshot dibersihkan saat logout dan 401.

```
final prefs = SharedPreferencesAsync();
await prefs.setString(
  'inspection_cache',
  jsonEncode({
    'fetchedAt': DateTime.now().toUtc()
        .toIso8601String(),
    'items': items.map((e) => e.toJson()).toList(),
  }),
);
```

Latihan terarah: Simpan cache setelah GET lalu baca ulang. Pastikan fetchedAt tidak berubah hanya karena snapshot ditampilkan kembali.

55. Kebijakan fallback yang terbatas

Repository mencoba API terlebih dahulu. Pada GET sukses, ia mengembalikan data server dan memperbarui snapshot. Jika penulisan cache gagal setelah GET sukses, data server tetap merupakan hasil valid; kegagalan cache dicatat terpisah.

Gangguan koneksi atau timeout GET dapat memakai cache valid. Hasil diberi isCached dan fetchedAt agar controller serta UI menunjukkan mode read-only. Tidak ada POST yang disimpan ke antrean saat offline.

401 selalu menutup session dan membersihkan cache. 500, 422, serta JSON invalid ditampilkan sebagai error menurut kebijakan materi sumber. Ini mencegah snapshot menutupi masalah server atau kontrak yang perlu diperbaiki.

Pisahkan exception transport dari exception status dan parsing. Menangkap semua exception lalu mengembalikan cache akan menghilangkan perbedaan tersebut. Repository juga perlu memastikan session masih sama sebelum menulis snapshot hasil request yang terlambat. Detail generation dibahas pada halaman berikutnya.

Hasil GET	Kebijakan
200 valid	Server data + update cache
200, cache write gagal	Server data + warning storage
Network / timeout	Cache valid, read-only
401	Clear session/cache
500 / JSON invalid	Error tanpa fallback
Cache rusak	Abaikan/hapus; error jelas

Latihan terarah: Uji fail-list dan stop server secara terpisah. Hanya gangguan transport yang boleh menggunakan snapshot.

56. Startup, logout, dan hasil lama

Pada startup, token tidak ada berarti login. Token ada berarti aplikasi mencoba GET server untuk memeriksa session. Jika server mengembalikan 401, token dan cache dibersihkan. Cache tidak dibuka tanpa konteks session yang sesuai.

Logout bukan sekadar pindah layar. Tandai session lama tidak aktif, tutup controller yang dimilikinya, hapus token serta snapshot, lalu tampilkan login. Async yang sudah berjalan dapat selesai setelah logout sehingga hasilnya harus diabaikan.

Gunakan generation session atau dependency scope yang dapat dibatalkan secara logis. Ambil generation sebelum request; sebelum commit state atau menulis cache, periksa masih sama. Menutup controller saja melindungi UI, tetapi repository lama juga tidak boleh menulis cache setelah pembersihan.

Perubahan session dan write cache perlu dikoordinasikan. Implementasi produksi dapat memakai scope cancellation serta urutan operasi storage yang terserialisasi. Pada latihan, tunjukkan race dengan mock delay dan buktikan bahwa record session lama tidak muncul kembali setelah logout.

```
int sessionGeneration = 0;

// Sebelum operasi:
final expected = sessionGeneration;
final result = await api.list();
if (expected != sessionGeneration) return;
// Commit hanya pada session yang masih aktif.

// Saat logout, sebelum clear storage:
sessionGeneration++;
controller.dispose();
await sessionStore.clear();
await cache.clear();
```

Latihan terarah: Mulai load lambat lalu logout. Setelah response tiba, login screen dan cache tetap bersih; data lama tidak boleh kembali.

57. Praktikum session dan cache

Input: login, GET, dan POST sudah lulus. Buat session store dan inspection cache. Tambahkan fetchedAt serta isCached pada hasil repository. Controller membaca metadata dan UI menunjukkan banner cache read-only.

Langkah 1: GET sukses lalu restart app. Token dibaca dan GET baru memvalidasi session. Langkah 2: matikan server setelah snapshot tersimpan. App menampilkan snapshot valid dengan waktu asal dan create disabled.

Langkah 3: gunakan token salah untuk memicu 401. Token serta cache dibersihkan dan login muncul. Langkah 4: isi cache dengan JSON rusak; baca ulang tanpa crash dan tanpa success palsu. Langkah 5: logout saat load lambat untuk menguji hasil session lama.

Uji pula penulisan cache gagal setelah GET sukses. Daftar server tetap ditampilkan dengan catatan storage; jangan melaporkan GET gagal jika response sebenarnya valid.

Bukti selesai: restart tervalidasi, offline berlabel, timestamp akurat, read-only, 401 bersih, cache rusak aman, dan race logout terkendali. Tulis status setiap skenario secara terpisah, bukan satu tanda centang “cache bekerja”.

Skenario	Acceptance
GET ulang startup	Session benar-benar diuji
Server mati	Snapshot berlabel dan read-only
401	Token/cache hilang; login
Cache rusak	Tidak crash; tidak success palsu
Logout saat load	Hasil lama diabaikan

58. Search, filter, dan pagination

Search lokal menggunakan daftar yang sudah dimuat. Normalisasi query dengan trim dan lowercase agar pencarian tidak bergantung pada spasi tepi atau perbedaan huruf besar. Kode aset atau lokasi dapat menjadi field pencarian.

Filter condition diterapkan bersama search. Jangan menimpa daftar sumber dengan visibleItems; pengguna harus dapat menghapus query untuk melihat data semula. Empty hasil filter memakai pesan “Tidak ada hasil yang cocok”, bukan “Server gagal”.

Jika API menggunakan pagination, search lokal hanya mencari items yang telah dimuat. UI perlu menjelaskan batas tersebut atau menggunakan pencarian server yang memiliki kontrak query. total server membantu mengetahui apakah hasil masih sebagian.

Pagination penuh menjadi pengayaan. Konsep yang didemonstrasikan meliputi page, per_page, urutan stabil, penghentian load-more, dan deduplication ID. Ketika data baru masuk, offset pagination dapat berubah; desain produksi perlu memilih strategi yang cocok dengan server.

```
final q = search.trim().toLowerCase();
final visible = items.where((item) {
  final matches = item.assetCode.toLowerCase()
      .contains(q) || item.location.toLowerCase()
      .contains(q);
  return matches && (filter == null ||
      item.condition == filter);
}).toList();
```

Latihan terarah: Cari dengan huruf besar dan spasi tepi, lalu aktifkan filter. Hapus semua filter dan periksa data sumber tetap lengkap.

59. Diagnosis koneksi dan parsing

Diagnosis dimulai dari lapisan terendah yang belum terbukti. Pastikan proses server hidup, host dan port benar, lalu route target dapat mencapai health. Baru setelah response diterima, periksa status dan bentuk JSON.

Connection refused biasanya menunjukkan tidak ada listener yang dapat menerima koneksi pada alamat tersebut. Timeout dapat berasal dari route, firewall, server lambat, atau response yang belum sampai. Type cast error menunjukkan data sudah diterima tetapi kontrak parsing tidak cocok.

Cleartext blocked berarti kebijakan Android menolak HTTP. Periksa konfigurasi debug dan full run. TLS error pada release diperiksa melalui host staging serta sertifikat, bukan dengan mengubah validator model.

Jika state berubah tetapi UI tetap, trace controller emit dan listener. Jika request ganda, cari load dari build atau callback berulang. Pisahkan log status jaringan dari log input dan jangan mencetak authorization header. Bukti diagnosis harus menyebut gejala yang benar, sebab yang ditemukan, dan skenario uji ulang.

Gejala	Pemeriksaan awal
Refused	Proses, bind address, port
Timeout	LAN, firewall, delay server
Cleartext blocked	Manifest debug, full run
Parsing error	Body/items/field/timestamp
UI diam	Mutation, emit, listener
Data lama kembali	Session ownership dan generation

Latihan terarah: Pilih dua gejala dan demonstrasikan perbaikan berdasarkan lapisan, tanpa mengubah bagian yang tidak berkaitan.

60. Praktikum matriks kegagalan

Siapkan tabel sebelum menjalankan skenario. Catat konfigurasi mock, aksi pengguna, status yang diharapkan, hasil aktual, dan bukti. Hentikan server lama sebelum menjalankan flag baru agar port tidak dipakai dua proses.

Skenario 1: token invalid menghasilkan 401 dan clear session. Skenario 2: POST note pendek dari REST client menghasilkan 422; pada Flutter, validator lokal mencegah kasus itu, sedangkan error server yang dipaksakan tetap mempertahankan form.

Skenario 3: --fail-list menghasilkan 500 tanpa fallback. Skenario 4: --fail-create menghasilkan error simpan tanpa menghapus input. Skenario 5: --delay 11 menunjukkan GET timeout dan POST hasil belum pasti. Skenario 6: stop server setelah GET sukses menampilkan cache berlabel.

Tambahkan search/filter saat data server dan saat cache. Create tetap disabled pada cache. Uji logout ketika delay aktif agar race session terlihat. Setiap skenario dipulihkan ke kondisi normal sebelum skenario berikutnya.

Bukti selesai: semua status dibedakan, recovery dapat dijalankan, serta pesan UI sesuai makna. Kegagalan yang belum diperbaiki dicatat dengan PIC dan langkah lanjut sebelum checkpoint.

Skenario	Hasil wajib
401	Session dibersihkan
422	Input dipertahankan
500	Error tanpa fallback
GET timeout	Cache valid atau error
POST timeout	Hasil belum pasti
Offline	Read-only berlabel

61. Handover dan review day-2

Demo hari kedua dimulai dengan login salah dan benar, lalu membaca data server. Create valid menghasilkan ID server, form invalid ditolak, dan klik simpan berulang tidak membuat operasi paralel. Tunjukkan detail record baru.

Selanjutnya demonstrasikan error, retry, cache offline, timestamp, serta logout. Server 401 harus membersihkan session. Hasil request session lama tidak boleh kembali mengisi daftar atau snapshot setelah logout.

README memuat cara menjalankan mock API, alamat emulator/device, dependency, dan data akun dummy. Jangan menyimpan token real atau credential produksi. Sertakan matriks uji serta daftar kendala sebelum foto dan release hari ketiga.

Review pemahaman membahas perbedaan service, repository, dan controller; alasan 401 tidak memakai cache; ketidakpastian POST timeout; serta arti fetchedAt. Jawaban perlu ditunjukkan pada source atau demo, bukan hanya hafalan istilah.

```
git status
git add .
git diff --cached
git commit -m "Complete day 2 API and session"
git tag day-2
```

Kelompok	Bukti lulus
Login/restart	Token dibaca; GET memvalidasi
GET/POST	Kontrak valid; ID muncul sekali
State/error	Feedback dan recovery sesuai
Cache	Timestamp, read-only, rusak aman
Logout/401	Token/cache bersih; hasil lama aman

62. Plugin dan konfigurasi native

Plugin menyediakan API Dart untuk fungsi platform, misalnya memilih foto. Di balik API tersebut, terdapat implementasi Android atau iOS dan dependency native. Menambah package dapat memerlukan perubahan toolchain atau konfigurasi platform.

Baca README versi yang benar-benar terpasang, bukan tutorial versi lama. Minimum SDK, permission, backup, dan perilaku lifecycle dapat berubah. Setelah menambah plugin, lakukan full run. Hot reload tidak cukup untuk membangun implementasi native baru.

Gunakan plugin yang sesuai sebelum menulis platform channel sendiri. MethodChannel berguna jika capability yang dibutuhkan tidak tersedia atau memerlukan integrasi khusus. Ia menambah tanggung jawab pada kontrak, error, lifecycle, dan pengujian dua bahasa.

Pada training, satu fitur foto menjadi praktik platform wajib. Fitur lokasi, notifikasi, dan file tidak semuanya ditambahkan agar waktu cukup untuk pengujian serta release. Kualitas satu integrasi yang dibuktikan lebih mudah dinilai daripada banyak fitur yang belum diuji.

```
flutter pub add image_picker

# Setelah dependency/native berubah:
flutter pub get
flutter run

# Hentikan run lama terlebih dahulu bila diperlukan.
```

Latihan terarah: Catat versi image_picker yang tersimpan pada lockfile dan minimum SDK dari README versi tersebut.

63. Kontrak foto lokal dan ID inspeksi

Foto pada latihan merupakan preview lokal yang berhubungan dengan sebuah record. Inspection ID adalah identitas hubungan tersebut. Posisi item dalam daftar tidak boleh menjadi pengganti ID karena posisi berubah setelah search, filter, atau refresh.

Panel foto pada detail menerima inspectionId dan ValueKey berdasarkan ID. Jika berpindah dari record A ke B, state foto A tidak boleh menempel pada B. Bila implementasi menyimpan foto di memory, gunakan mapping ID ke foto dengan ownership yang jelas.

XFile dari picker dapat menunjuk file sementara. Preview yang berhasil tidak membuktikan persistensi setelah restart dan tidak membuktikan upload. Buku mempertahankan batas ini sesuai materi sumber agar hasil training tidak disalahartikan.

Pengembangan lanjut memerlukan penyalinan file ke lokasi aplikasi yang permanen, metadata attachment, ukuran, retensi, serta upload multipart. Jangan menambahkan path foto sementara ke cache seolah ia merupakan attachment server yang dapat dibuka selamanya.

```
PhotoPanel(
  key: ValueKey(item.id),
  inspectionId: item.id,
)

// Kontrak panel:
// Input: ID record aktif.
// Output: preview lokal, belum upload.
```

Latihan terarah: Pilih foto pada record A lalu buka B. Pastikan B tidak menunjukkan foto A dan label tidak mengklaim upload sukses.

64. Memilih galeri atau kamera

ImagePicker.pickImage menghasilkan XFile? yang dapat null ketika pengguna membatalkan. Pilihan source menentukan apakah galeri atau kamera dibuka. Parameter ukuran dan kualitas membantu membatasi preview, tetapi perlu diuji pada file nyata.

Sebelum membuka picker, aktifkan busy guard. Setelah selesai, periksa mounted dan identitas record yang masih aktif. Jika hasil null, pertahankan foto sebelumnya. Jangan mengganti foto menjadi kosong hanya karena pengguna menekan cancel.

Pada Android, Image.file dapat membaca path lokal melalui dart:io. Pola ini tidak berlaku langsung untuk web. Karena training berfokus Android, contoh tidak memaksakan abstraksi semua platform pada satu sesi.

Tangani file unavailable melalui errorBuilder atau pemeriksaan file. Pengguna menerima pesan dan aksi pilih ulang. File besar dapat menambah biaya decode serta memory; ukuran layout saja tidak selalu membatasi ukuran gambar yang didecode.

```
final picker = ImagePicker();
final result = await picker.pickImage(
  source: ImageSource.gallery,
  maxWidth: 1600,
  imageQuality: 80,
);
if (!mounted || result == null) return;
setState(() => photo = result);

// import dart:io;
Image.file(
  File(photo!.path), height: 200,
  errorBuilder: (_, __, ___) =>
      const Text('Foto tidak tersedia; pilih ulang'),
)
```

Latihan terarah: Pilih foto, buka picker lagi, lalu cancel. Foto pertama tetap terlihat.

65. Cancel, error, dan operasi busy

Cancel merupakan hasil normal dari interaksi pengguna. Picker null tidak perlu dilaporkan sebagai kegagalan sistem. PlatformException berbeda: dapat berkaitan dengan plugin, kemampuan perangkat, permission, atau konfigurasi.

Gunakan busy flag untuk mencegah dua picker dibuka bersamaan. Pada finally, akhiri busy hanya jika State masih hidup. Tombol diberi feedback agar pengguna mengetahui operasi sedang berlangsung.

Jika halaman sudah ditutup ketika picker selesai, jangan setState. Jika record aktif berubah, jangan memasang hasil ke record lain. Capture ID sebelum operasi dan cocokkan dengan konteks yang dituju ketika hasil kembali.

Penolakan akses atau fitur unavailable perlu pesan yang sesuai dengan hasil plugin. Jangan menambahkan permission lama secara acak sebagai solusi semua kasus. Android Photo Picker dan kamera dapat mempunyai alur berbeda; baca dokumentasi paket serta uji versi Android yang digunakan di kelas.

Kondisi	Perilaku
Result null	Pertahankan foto lama
PlatformException	Pesan dan aksi coba lagi
Busy	Disable aksi picker baru
State selesai	Abaikan update UI
ID berubah	Jangan pindahkan attachment
File hilang	Tampilkan pilih ulang

Latihan terarah: Uji cancel, klik ganda, dan kembali dari detail saat operasi. Catat hasil nyata sesuai perangkat.

66. Recovery lost data pada startup

Android dapat menghentikan activity ketika aplikasi berada di background saat picker aktif. Hasil tidak selalu kembali ke Future yang memulai operasi. retrieveLostData digunakan untuk memulihkan hasil saat aplikasi dibuka kembali.

Pemulihan diperiksa pada startup Android oleh pemilik alur foto atau session, bukan hanya ketika detail tertentu kebetulan dibuka. Sebelum picker, simpan pending inspection ID. Setelah restart, baca lost data dan pending ID untuk menentukan record tujuan.

Jika ID tujuan belum tersedia karena daftar belum dimuat, tahan hasil recovery dalam state pemilik dan tampilkan setelah identitas dikonfirmasi. Jangan mengambil hasil lost data pada detail B lalu memasangnya ke B hanya karena itulah layar yang sedang aktif.

Tangani response exception dan hasil tanpa pending ID sebagai kasus terpisah. Setelah konsumsi berhasil atau dinyatakan gagal, bersihkan penanda sesuai kebijakan. Penanda operasi normal juga dibersihkan setelah picker selesai agar tidak mengaitkan foto berikutnya dengan record lama.

```
// Sebelum picker dibuka:
await prefs.setString(
  'pending_photo_inspection', inspectionId,
);

// Di alur startup Android:
final pendingId = await prefs.getString(
  'pending_photo_inspection',
);
final lost = await picker.retrieveLostData();
// Cocokkan pendingId dengan record tujuan.
// lost.files => simpan hasil untuk ID itu.
// lost.exception => pesan recovery gagal.
```

Latihan terarah: Demonstrasikan jalur recovery pada perangkat atau melalui fake. Foto tidak boleh ditempelkan ke ID yang berbeda dari pending ID.

67. MethodChannel sebagai pengayaan

MethodChannel mengirim request antara Dart dan implementasi platform. Nama channel harus sama pada kedua sisi. Nama method dan shape argument juga merupakan kontrak yang perlu dijaga.

invokeMethod mengembalikan Future dan dapat menghasilkan PlatformException. Handler Kotlin memeriksa method yang didukung, menyelesaikan result satu kali, atau memberikan notImplemented. Data yang dikirim harus cocok dengan codec yang digunakan.

Gunakan contoh kecil seperti membaca string informasi platform. Jangan menambahkan operasi perangkat kompleks pada saat peserta belum menyelesaikan foto dan test. Setiap channel custom memerlukan pengujian native, lifecycle, serta penanganan thread sesuai pekerjaannya.

Tujuan demo adalah menunjukkan bahwa Flutter tetap dapat berinteraksi dengan Android, bukan mengganti semua plugin dengan kode sendiri. Setelah memahami kontrak, tim dapat mengevaluasi kebutuhan integrasi khusus perusahaan pada sesi lanjutan.

```
// Dart: contoh fragmen
const channel = MethodChannel('training/platform');
final label = await channel.invokeMethod<String>(
  'platformLabel',
);

// Kotlin: di handler MethodChannel
if (call.method == "platformLabel") {
    result.success("Android training")
} else {
    result.notImplemented()
}
```

Latihan terarah: Jelaskan apa yang terjadi bila nama channel atau method berbeda. Tetapkan error UI yang masuk akal untuk demo tersebut.

68. Praktikum foto Android

Input: checkpoint day-2 dan detail record server sudah benar. Tambahkan image_picker, lakukan full run, dan buat PhotoPanel yang menerima inspectionId. Tetapkan ownership foto serta recovery pada alur startup.

Langkah 1: simpan pending ID sebelum memilih foto. Langkah 2: tambahkan galeri dan kamera sesuai perangkat, busy guard, mounted check, dan handling cancel. Langkah 3: tampilkan preview dengan pesan file unavailable.

Langkah 4: pilih foto pada record A lalu pindah ke B. Uji juga perubahan urutan daftar melalui filter. Langkah 5: periksa retrieveLostData pada startup dan mapping hasil terhadap pending ID. Gunakan fake untuk jalur sulit direproduksi, lalu catat bagian yang tetap memerlukan device test.

Langkah 6: cancel ketika foto lama tersedia, kemudian uji kembali. Bersihkan pending ID setelah operasi normal selesai. Jangan menghapus data recovery sebelum hasil dipetakan dengan benar.

Bukti selesai: screenshot preview, cancel mempertahankan foto, busy guard, file unavailable, serta recovery terikat ID. Tulis batas bahwa foto belum persistent dan belum di-upload. Simpan commit sebelum pengujian kualitas.

Skenario	Acceptance
Pick	Preview pada ID aktif
Cancel	Foto sebelumnya tetap
Record lain	Tidak membawa foto A ke B
Error	Pesan; app tetap berjalan
Recovery	Pending ID dihormati

69. Unit, widget, dan device test

Unit test memeriksa aturan atau parsing tanpa membangun seluruh aplikasi. Validator note, condition, dan timestamp cocok untuk unit test. Hasilnya cepat dan dapat dijalankan berulang saat refactor.

Widget test membangun widget dalam lingkungan test dan mensimulasikan interaksi Flutter. Ia dapat memeriksa pesan validator, tombol retry, loading, serta navigation. Plugin native dan jaringan sebaiknya diganti dependency terkendali untuk test tersebut.

Device smoke test memeriksa integrasi nyata seperti permission, kamera, jaringan release, storage, dan instalasi APK. Unit serta widget test yang lulus tidak menggantikan pemeriksaan ini karena lingkungan serta konfigurasi platform berbeda.

Pilih assertion yang menyatakan perilaku, bukan implementasi internal yang kebetulan dipakai sekarang. Misalnya, “form tetap terbuka dengan pesan field wajib” lebih bermakna daripada “method validator dipanggil satu kali”. Test harus gagal ketika perilaku penting rusak.

Jenis	Contoh bukti
Unit	needs_action menolak note 9 karakter
Parsing	Timestamp UTC dan field wajib
Widget	Save invalid tidak menutup form
Controller	Error ke retry ke success
Device release	Login, foto, cache dan logout nyata

Latihan terarah: Kelompokkan lima acceptance proyek ke jenis test. Tandai yang wajib tetap diuji pada Android nyata.

70. Unit test batas validator

Test validator memeriksa nilai batas, bukan hanya contoh yang jelas valid. Untuk kode aset, uji kosong, spasi, 30 karakter, dan 31. Untuk note temuan, uji 9 dan 10 setelah trim, lalu 500 dan 501.

Pisahkan test good yang mengizinkan note kosong dari needs_action yang mewajibkan catatan. Jika kedua kondisi digabung tanpa nama yang jelas, test gagal sulit menjelaskan aturan mana yang berubah.

Import validator dari package dpl_inspection sesuai nama proyek pada pubspec. Fungsi murni tidak membutuhkan setup device. Gunakan test name yang menggambarkan kontrak sehingga output test dapat dibaca sebagai daftar aturan.

Demonstrasikan bahwa test dapat gagal: ubah batas minimum menjadi nilai yang salah, jalankan test, lalu pulihkan. Tujuan latihan ini memastikan test benar-benar melindungi perilaku. Jangan menyimpan perubahan yang sengaja merusak pada checkpoint final.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:dpl_inspection/validators.dart';

void main() {
  test('needs_action rejects 9 characters', () {
    expect(validateNote('needs_action', '123456789'),
        isNotNull);
  });
  test('needs_action accepts 10 characters', () {
    expect(validateNote('needs_action', '1234567890'),
        isNull);
  });
  test('good permits empty note', () {
    expect(validateNote('good', ''), isNull);
  });
}
```

Latihan terarah: Tambah test spasi tepi dan batas 500. Rusakkan satu aturan, buktikan fail, lalu pulihkan hingga pass.

71. Unit test kontrak model

Parsing yang defensif dapat diuji dengan payload valid dan beberapa variasi rusak. Periksa ID, kode aset, condition, note, serta createdAt.isUtc pada payload yang mengikuti kontrak.

Tambahkan kasus field wajib hilang, tipe salah, condition tidak dikenal, dan timestamp tanpa zona atau tidak dapat diparse. Model lampiran melempar FormatException pada input kontrak yang tidak valid. Test diselaraskan dengan kebijakan tersebut, berbeda dari cast sederhana pada slide.

Round-trip toJson dan fromJson memeriksa kesesuaian serialisasi cache, tetapi tidak cukup sebagai satu-satunya test. Dua method yang sama-sama salah dapat terlihat konsisten. Tetap gunakan fixture yang ditulis sesuai kontrak server secara independen.

Jangan membuat test mengharapkan string timestamp persis ketika tujuan sebenarnya adalah waktu UTC yang sama. Pilih assertion yang melindungi makna data. Jika error payload harus muncul pada UI, tambahkan test controller yang mengubah parsing failure menjadi error state.

```
test('parses server identity and UTC', () {
  final x = Inspection.fromJson({
    'id': '1', 'assetCode': 'PMP-001',
    'location': 'Workshop A', 'condition': 'good',
    'note': '',
    'createdAt': '2026-10-06T01:30:00Z',
  });
  expect(x.id, '1');
  expect(x.createdAt.isUtc, isTrue);
});

test('rejects missing required fields', () {
  expect(() => Inspection.fromJson({'id': '1'}),
      throwsA(isA<FormatException>()));
});
```

Latihan terarah: Tambah satu fixture condition salah dan satu createdAt invalid. Pastikan error tidak diubah menjadi record kosong.

72. Widget test form invalid

Widget test memulai dari tampilan yang benar-benar digunakan pengguna. Bangun MaterialApp dengan InspectionForm, temukan tombol simpan, pastikan visible, lalu tap. Form kosong seharusnya menampilkan pesan dan tetap terbuka.

Key stabil membantu menemukan field atau tombol tanpa bergantung pada posisi visual. Gunakan key seperti saveButton, assetField, dan locationField. Key test tidak menggantikan label yang jelas bagi pengguna.

pump membangun frame berikutnya, sedangkan pumpAndSettle menunggu animasi atau pekerjaan yang dapat selesai. Loading yang berjalan tanpa akhir dapat membuat settle timeout. Gunakan Future terkendali untuk menguji busy state, bukan delay tidak deterministik.

Test form valid dapat mengisi field, memilih condition, menekan simpan, dan memeriksa hasil navigation melalui halaman host. Error API diuji dengan fake create yang gagal sehingga input tetap ada. Hindari mengirim request ke mock nyata pada widget test dasar.

```
await tester.pumpWidget(
  const MaterialApp(home: InspectionForm()),
);
final save = find.byKey(const Key('saveButton'));
await tester.ensureVisible(save);
await tester.tap(save);
await tester.pump();
expect(find.text('Kode aset wajib diisi'),
    findsOneWidget);
expect(find.byType(InspectionForm), findsOneWidget);
```

Latihan terarah: Tambah key saveButton pada form. Uji bahwa invalid save tidak menghasilkan result navigation.

73. Fake repository dan state async

Fake dependency memungkinkan controller diuji tanpa jaringan. Repository palsu dapat mengembalikan tiga record, list kosong, error, atau Future yang dikendalikan melalui Completer. Dengan demikian loading dapat diperiksa sebelum operasi selesai.

Test load dimulai dengan isLoading false, memanggil load, lalu memeriksa true. Selesaikan fake Future dan periksa items serta isLoading false. Pada error, periksa message dan kemampuan retry. Empty tetap menjadi hasil valid.

Untuk submit guard, panggil create dua kali sebelum Future pertama selesai. Fake mencatat jumlah operasi; hanya satu yang berjalan. Setelah selesai, record menggunakan ID yang diberikan fake server dan tombol kembali siap.

Uji dispose atau logout ketika Future belum selesai. Tidak ada notifikasi UI setelah scope berakhir dan hasil lama tidak boleh menulis snapshot. Bila repository menyimpan cache, fake storage dapat mengungkap side effect yang tidak terlihat melalui controller saja.

```
final completer = Completer<List<Inspection>>();
final fake = FakeRepository(completer.future);
final controller = InspectionController(fake);

final operation = controller.load();
expect(controller.isLoading, isTrue);
completer.complete([sampleInspection]);
await operation;
expect(controller.isLoading, isFalse);
expect(controller.items.single.id, 'server-1');

// FakeRepository mengikuti interface proyek.
```

Latihan terarah: Buat fake yang gagal sekali lalu sukses pada retry. Assertion harus memeriksa transisi, data, dan pesan yang dibaca pengguna.

74. Quality gate sebelum release

Format menjaga gaya kode konsisten. Analyze menemukan error serta masalah statis. Test memeriksa kontrak perilaku yang telah ditulis. Ketiganya merupakan pemeriksaan berbeda; format yang selesai tidak berarti aplikasi telah diuji.

Jalankan dari root proyek. Jika test template counter masih mengharapkan aplikasi awal, ganti dengan test yang sesuai UI sekarang. Jangan menghapus test bermakna hanya untuk membuat hasil terlihat hijau.

Baca kegagalan pertama dan cari sebab. Satu parsing error dapat membuat beberapa test lanjutan gagal. Perbaiki sumbernya lalu ulangi pemeriksaan relevan. Setelah seluruh pemeriksaan lulus, simpan output ringkas dan commit kualitas.

Quality gate tidak menggantikan smoke test release. Permission, API environment, signer, kamera, dan storage tetap diperiksa melalui APK. Pada dokumen handover, pisahkan hasil analyze/test dari hasil device agar status yang dilaporkan akurat.

```
dart format lib test
flutter analyze
flutter test

# Bila ingin coverage sebagai tambahan:
flutter test --coverage
```

Latihan terarah: Catat tanggal, commit, dan hasil tiga pemeriksaan. Laporkan blocker yang tersisa sebelum build release.

75. DevTools dan profile mode

DevTools menyediakan alat untuk membaca widget tree, menelusuri logic, mengamati frame, dan memeriksa memory. Pilih alat sesuai gejala: overflow melalui Inspector, aturan melalui debugger, jank melalui Performance, dan resource melalui Memory.

Pengukuran performa dilakukan dalam profile pada physical Android yang sama. Debug memiliki overhead dan tidak mewakili release secara langsung. Emulator tetap berguna untuk UI, tetapi bukan baseline yang kuat untuk kesimpulan performa perangkat.

Tetapkan skenario sebelum merekam, misalnya scroll 200 record atau mengetik query yang sama. Catat model device, refresh rate, jumlah data, ukuran foto, mode build, serta commit. Tanpa kondisi yang sama, perbandingan sebelum dan sesudah dapat menyesatkan.

Frame budget bergantung pada refresh rate: sekitar 16,7 ms pada 60 Hz dan 8,3 ms pada 120 Hz. Gunakan angka sebagai konteks diagnosis, bukan target universal yang diasumsikan untuk semua perangkat. Bedakan latency jaringan dari pekerjaan render dan parsing.

```
flutter run --profile -d ID_DEVICE

# Jika menggunakan API staging, satu baris:
# flutter run --profile -d ID_DEVICE --dart-
define=API_BASE_URL=https://HOST_STAGING
```

Latihan terarah: Rekam satu skenario scroll yang dapat diulang. Simpan screenshot trace serta kondisi pengukuran.

76. Mengukur satu perbaikan

Mulai dengan baseline sebelum mengubah kode. Catat gejala yang dapat diulang, lalu rekam trace. Jika daftar lambat, periksa apakah semua child dibuat sekaligus, filter dihitung berulang, gambar terlalu besar, atau request dipicu dari build.

Pilih satu penyebab yang didukung bukti. Contoh perbaikan dapat berupa ListView.builder, pemisahan komponen listener, penghitungan visibleItems saat input berubah, atau preview dengan ukuran decode yang sesuai. const membantu pada konteks tertentu, tetapi bukan jawaban untuk semua jank.

Ukur ulang pada device, jumlah data, dan skenario sama. Jangan mengubah beberapa variabel lalu menyimpulkan satu perubahan menyebabkan seluruh peningkatan. Simpan commit dan bukti sesudah.

Kesimpulan menyebut dampak serta batas. Jika trace membaik pada skenario tertentu, laporkan skenario tersebut. Tidak perlu mengklaim “aplikasi 2x lebih cepat” tanpa ukuran yang mendukung. Bila perubahan tidak membantu, pulihkan atau evaluasi hipotesis lain.

Tahap	Catatan yang disimpan
Baseline	Device, skenario, data, trace
Diagnosis	Pekerjaan yang diduga mahal
Perubahan	Satu sebab dan commit
Ukur ulang	Kondisi sama; trace sesudah
Kesimpulan	Dampak teramati dan batas

Latihan terarah: Isi lembar profiling di lampiran. Satu perubahan harus mempunyai bukti sebelum serta sesudah.

77. Gambar dan resource memory

Gambar beresolusi tinggi dapat membutuhkan memory decode jauh lebih besar daripada ukuran file terkompresi. Menampilkan image dengan height 200 tidak otomatis berarti decoder hanya memakai data setinggi itu.

Gunakan parameter picker dan opsi decode yang sesuai jika dibutuhkan. Periksa kualitas visual serta kemampuan membaca temuan pada foto. Mengurangi resolusi secara berlebihan dapat menghilangkan informasi yang diperlukan pengguna.

Memory juga dipengaruhi object yang tidak dilepas: controller, listener, subscription, dan client. Buka serta tutup layar berulang sambil mengamati gejala. Garbage collector tidak menggantikan disposal resource yang memiliki lifecycle eksplisit.

Bedakan cache gambar yang dikelola framework dari kebocoran object aplikasi. Peningkatan memory sementara tidak selalu berarti leak. Diagnosis memerlukan pola, retensi, dan skenario yang dapat diulang. Pada training, cukup pilih satu kasus dan jelaskan bukti yang mendukung perbaikan.

```
Image.file(
  File(photo.path),
  height: 200,
  fit: BoxFit.cover,
  cacheWidth: 800,
  errorBuilder: (_, __, ___) =>
      const Text('Foto tidak tersedia'),
)
```

Latihan terarah: Bandingkan preview foto besar sebelum/sesudah pembatasan decode. Catat kualitas visual, memory, dan batas pengukuran.

78. Praktikum testing dan profiling

Langkah 1: tulis unit test validator dan parsing dari kontrak. Tambahkan kasus batas serta payload rusak. Langkah 2: tulis widget test form invalid dengan key yang stabil dan assertion bahwa form tetap terbuka.

Langkah 3: rusakkan satu aturan secara sengaja, jalankan test, lalu pulihkan. Bukti fail dan pass menunjukkan test melindungi perilaku. Langkah 4: gunakan fake repository untuk error/retry atau submit guard.

Langkah 5: jalankan format, analyze, dan test. Setelah hasil bersih, pilih skenario scroll, search, atau preview gambar pada device nyata. Rekam baseline profile, lakukan satu perbaikan, dan ukur ulang dengan kondisi sama.

Jangan memakai waktu pengujian untuk menambah fitur baru. Fokus menyelesaikan risiko yang sudah ditemukan pada alur inti. Simpan screenshot DevTools, nama device, mode, data, dan commit sebelum/sesudah.

Bukti selesai: unit serta widget test pass, satu demonstrasi fail relevan, dan catatan profiling yang dapat diperiksa. Bila physical device belum tersedia, tandai profiling terhalang prasyarat, tanpa mengisi angka performa yang tidak diukur.

Kegiatan	Bukti
Validator	Test batas pass
Parser	Fixture valid/rusak pass
Widget	Invalid tidak menutup form
Mutation demo	Fail lalu pass
Profile	Trace sebelum/sesudah

79. Polishing alur end-to-end

Sebelum signing, jalankan alur lengkap dari login sampai logout. Gunakan data dummy yang mudah dikenali. Periksa apakah UI selalu menjelaskan operasi sedang berjalan, gagal, atau memakai snapshot.

Uji invalid dan valid create, submit guard, detail, search/filter, error/retry, cache read-only, serta foto. Input tidak hilang ketika POST gagal. POST sukses tidak dilaporkan gagal hanya karena GET refresh berikutnya bermasalah.

Periksa layar sekitar 360 logical pixels, landscape, keyboard, note panjang, dan font besar. Status harus memiliki teks selain warna. Detail dan form dapat di-scroll. Tombol penting tetap terjangkau.

Prioritaskan bug berdasarkan dampak: salah record, kehilangan input, duplikasi, session lama, dan crash lebih mendesak daripada dekorasi. Simpan daftar bug yang telah diperbaiki serta yang tersisa. Jangan memperluas scope menjelang release sehingga bukti kualitas yang sudah dikumpulkan menjadi tidak relevan.

Area	Pertanyaan review
Data	ID dan detail sesuai record?
Create	Validasi/guard/input aman?
Network	Error dan recovery sesuai?
Cache	Read-only dengan fetchedAt?
Foto	Cancel dan ID aman?
Session	Logout/401 membersihkan data?

Latihan terarah: Jalankan checklist acceptance lampiran dan simpan bukti untuk setiap kelompok inti.

80. Batas kode bersama dan toolchain iOS

Flutter memungkinkan banyak kode model, aturan, widget, serta data flow dipakai lintas platform. Proyek native, capability, permission, identifier, dan signing tetap mempunyai kebutuhan platform masing-masing.

Build iOS menggunakan Mac dan Xcode. Simulator membantu memeriksa UI, sementara fitur hardware tetap diuji pada perangkat yang sesuai. Plugin dapat memerlukan konfigurasi Info.plist serta penjelasan penggunaan kamera atau photo library.

Bundle ID mengidentifikasi aplikasi; Team, certificate, dan provisioning mendukung signing serta distribusi. Alur lanjut meliputi konfigurasi project, archive/build, upload App Store Connect, lalu TestFlight dan pemeriksaan platform.

Bagian ini dialokasikan dua puluh menit pada hari ketiga. Peserta tidak diminta membuat IPA atau menyelesaikan distribusi iOS selama training Android. Untuk sesi lanjutan, tim menyiapkan Mac, Xcode, akun, device, dan PIC distribusi sebelum memulai praktik.

Dapat dibagi	Perlu pemeriksaan platform
Model dan validator	SDK/toolchain native
Banyak widget	Permission dan capability
Repository/logic	Implementasi plugin
Banyak test Dart	Device dan signing
Kontrak API	Distribusi serta account access

Latihan terarah: Buat daftar prasyarat sesi iOS lanjutan tanpa mengurangi acceptance Android.

81. Identitas dan versi Android

Application ID membedakan aplikasi Android saat instalasi dan distribusi. Namespace serta package MainActivity berhubungan dengan kode native. Periksa konsistensi saat mengganti identitas proyek latihan.

Gunakan ID latihan yang jelas, misalnya com.example.dplinspection.training. Ini contoh untuk kelas, bukan ID produksi yang disepakati perusahaan. Pergantian ID dapat membuat aplikasi dianggap berbeda oleh Android.

Version pada pubspec mempunyai bentuk nama+nomor build, misalnya 0.3.0+3. Nomor build dinaikkan untuk distribusi baru sesuai kebutuhan. Catat version, commit, environment, dan signer bersama artefak agar hasil dapat dilacak.

Minimum SDK mengikuti kebutuhan aplikasi serta plugin yang benar-benar terpasang. Compile dan target SDK juga harus sesuai toolchain serta kebijakan distribusi saat pelaksanaan. Jangan menyalin nomor SDK lama tanpa memeriksa error build serta dokumentasi terkini.

```
# pubspec.yaml
version: 0.3.0+3

// android/app/build.gradle.kts
android {
    namespace = "com.example.dplinspection.training"
    defaultConfig {
        applicationId =
            "com.example.dplinspection.training"
    }
}
```

Latihan terarah: Catat application ID, versi, build number, SDK, dan commit pada lembar release.

82. Membuat keystore latihan

Android menggunakan signature untuk mengidentifikasi pihak yang menandatangani aplikasi. Training memakai keystore khusus latihan agar proses signing dapat dipelajari tanpa memakai key produksi perusahaan.

keytool tersedia pada instalasi Java yang digunakan lingkungan Android. Jika tidak ada pada PATH, temukan lokasi Java melalui flutter doctor -v lalu gunakan path keytool yang sesuai. Pada Windows, path dengan spasi perlu quote.

Perintah di bawah meminta password secara interaktif. Jangan menambahkan password nyata ke listing buku, log, atau repository. Simpan keystore serta konfigurasi rahasia di lokasi yang dikelola pemilik training.

Alias, path, dan fingerprint certificate dicatat tanpa password. Keberhasilan key generation belum membuktikan build release memakai key tersebut. Setelah konfigurasi Gradle, verifikasi signature APK dan bandingkan certificate dengan keystore latihan.

```
keytool -genkeypair -v -keystore training-upload.jks -storetype JKS
-keyalg RSA -keysize 2048 -validity 3650 -alias training

# Jalankan command sebagai satu baris.
# Password diminta secara interaktif.

keytool -list -v -keystore training-upload.jks -alias training
```

Latihan terarah: Buat key latihan dan catat alias serta SHA-256 certificate fingerprint tanpa menyertakan password.

83. Properties dan konfigurasi signing

Simpan referensi keystore pada android/key.properties. storeFile menunjuk file yang benar di mesin peserta. Pada Windows, gunakan forward slash seperti D:/Training/training-upload.jks atau escape backslash sesuai Properties.

Tambahkan import Properties dan FileInputStream pada build.gradle.kts. Load file properties sebelum blok android, lalu buat signingConfigs release. Build type release harus menunjuk konfigurasi release, bukan debug.

Sesuaikan perubahan dengan file Gradle yang dihasilkan SDK terpasang. Contoh memakai Kotlin DSL; proyek Groovy memiliki sintaks berbeda. Jangan membuat dua blok release yang saling menimpa tanpa memeriksa file akhir.

key.properties dan file key dikecualikan dari Git. README hanya menjelaskan cara konfigurasi, bukan menyimpan credential. Implementasi signing lengkap yang terbagi menjadi bagian sebelum dan di dalam android terdapat pada lampiran.

```
storePassword=ISI_DI_MESIN_LATIHAN
keyPassword=ISI_DI_MESIN_LATIHAN
keyAlias=training
storeFile=D:/Training/training-upload.jks

# Tambahkan pada .gitignore sesuai lokasi:
android/key.properties
*.jks
*.keystore
```

Latihan terarah: Periksa git diff dan status sebelum commit. File rahasia tidak ikut, sedangkan petunjuk konfigurasi tetap tersedia.

84. Build APK dan AAB

APK digunakan untuk instalasi langsung pada perangkat Android. AAB adalah paket distribusi yang digunakan proses bundle tooling atau Google Play untuk menghasilkan APK yang sesuai target. AAB tidak diinstal langsung memakai adb install.

Build dijalankan setelah quality gate dan signing diperiksa. Gunakan base URL HTTPS staging yang nyata untuk artefak smoke test jaringan release. Placeholder host harus diganti; compile sukses tidak membuktikan endpoint dapat dicapai.

Perintah panjang pada cetakan dapat terbungkus secara visual, tetapi dijalankan sebagai satu baris. Path hasil mengikuti struktur Flutter yang dipakai: APK utama biasanya berada pada build/app/outputs/flutter-apk/app-release.apk dan AAB pada build/app/outputs/bundle/release/app.aab.

Simpan log build, version, environment, commit, dan signer. Jika build gagal, baca error pertama yang menjadi sebab, bukan hanya baris “Gradle task failed”. Jangan mengubah banyak versi dependency sekaligus sebelum memahami kegagalan.

```
flutter build apk --release
--dart-define=API_BASE_URL=https://HOST_STAGING

flutter build appbundle --release
--dart-define=API_BASE_URL=https://HOST_STAGING

# Ganti HOST_STAGING dengan endpoint HTTPS nyata.
# Command dijalankan masing-masing satu baris.
```

Latihan terarah: Build kedua artefak dan catat lokasi, ukuran, versi, commit, serta environment pada lembar release.

85. Verifikasi signer dan instalasi

apksigner memeriksa signature APK serta menampilkan certificate. Bandingkan fingerprint dengan key latihan. APK yang memakai debug signer belum memenuhi tujuan latihan release signing walaupun dapat diinstal.

AAB merupakan bundle bertanda tangan yang dapat diperiksa melalui jarsigner. Catat hasil verifikasi dan certificate yang digunakan. Play App Signing dapat memakai app signing key yang berbeda dari upload key; jangan menyamakan keduanya saat membandingkan APK hasil Play.

adb install -r digunakan untuk memasang APK dan mencoba mengganti app yang kompatibel. Update dengan application ID sama tetapi signer berbeda dapat ditolak. Jangan langsung menghapus app yang mempunyai data penting; pada perangkat latihan gunakan app dummy dan jelaskan sebab konflik signature.

Tool apksigner berada pada Android SDK Build-Tools bila belum tersedia pada PATH. jarsigner dan keytool berasal dari JDK. Catat lokasi tool yang digunakan agar peserta lain dapat mengulangi verifikasi dengan artefak yang sama.

```
apksigner verify --verbose --print-certs
build/app/outputs/flutter-apk/app-release.apk

jarsigner -verify -verbose -certs
build/app/outputs/bundle/release/app.aab

adb install -r build/app/outputs/flutter-apk/app-release.apk

# Masing-masing satu baris; gunakan path tool
# SDK/JDK jika belum tersedia pada PATH.
```

Latihan terarah: Tunjukkan certificate APK sesuai keystore latihan, kemudian jalankan smoke test menggunakan APK tersebut.

86. Smoke test release dan distribusi

Smoke test dijalankan pada APK release karena konfigurasi dan optimasi dapat berbeda dari debug. Mulai dari instalasi, launch, login, GET, create, detail, foto, cache, serta logout. Catat device dan Android version.

Uji dengan HTTPS staging yang diketahui. Jika staging atau akun tidak siap, tandai skenario terkait terhalang prasyarat dengan PIC dan langkah lanjut. Build serta signature dapat tetap diperiksa, tetapi jangan melaporkan seluruh smoke test lulus.

Google Play Internal Testing dibahas sebagai jalur distribusi opsional. Pemilik akun menyiapkan akses, app record, upload key, serta formulir yang diperlukan pada saat pelaksanaan. Proses review eksternal tidak menjadi target selesai di kelas.

Release notes menyebut fitur yang benar-benar tersedia dan batas cache/foto. Preview lokal bukan upload, cache bukan antrean offline, dan mock memory bukan backend produksi. Catatan ini membantu tim yang melanjutkan proyek menentukan pekerjaan berikutnya.

Kelompok smoke	Hasil yang dicatat
Install/launch	APK yang sama berhasil dibuka
API	HTTPS/login/GET/POST nyata
State	Error/retry/cache read-only
Platform	Foto/cancel/file error
Session	Logout/401 bersih
Distribusi	Upload opsional atau blocker PIC

Latihan terarah: Isi Lulus, Perlu perbaikan, atau Terhalang prasyarat untuk setiap skenario; jangan mengisi satu status untuk seluruh aplikasi.

87. Troubleshooting build dan release

Gradle gagal perlu diperiksa dari error penyebab: JDK, SDK, dependency, konfigurasi Kotlin DSL, atau plugin. Baris akhir task failed hanya menunjukkan hasil proses, bukan diagnosis.

Key tidak ditemukan mengarah pada storeFile, path, alias, atau password lokal. Signer debug mengarah pada assignment build type. Periksa konfigurasi efektif lalu build ulang dan verifikasi certificate, bukan hanya nama file APK.

API yang bekerja pada debug tetapi gagal release dapat berkaitan dengan INTERNET, TLS, base URL, atau konfigurasi jaringan. Missing plugin setelah perubahan dependency memerlukan full run/build sebelum menyimpulkan paket rusak.

Gunakan catatan perubahan dan kembali ke checkpoint bila perlu. clean digunakan ketika artefak lama relevan dengan masalah, bukan sebagai solusi universal. Simpan log penyebab pertama serta hasil uji ulang agar handover menjelaskan perbaikan yang telah dilakukan.

Gejala	Tindakan terarah
Gradle gagal	Periksa error sebab, JDK/SDK/plugin
Key tidak ada	Verifikasi path serta alias lokal
Signer debug	Assignment release dan verify ulang
API release gagal	INTERNET, HTTPS, host environment
Missing plugin	Full run/build setelah native berubah
Update APK ditolak	Periksa ID dan certificate

Latihan terarah: Tulis satu contoh diagnosis build tanpa membuka password. Lampirkan error pertama dan tindakan yang benar-benar dilakukan.

88. Praktikum release dan handover akhir

Mulai dari source yang telah lulus acceptance Flutter. Periksa identitas, version, manifest, environment, serta signing latihan. Jalankan format, analyze, dan test sebelum membuat APK/AAB.

Langkah 1: build release dengan konfigurasi yang dicatat. Langkah 2: verifikasi certificate APK dan signature AAB. Langkah 3: install APK pada device latihan dan jalankan smoke test nyata. Langkah 4: catat blocker staging atau akun secara terpisah dengan PIC.

Langkah 5: buat release notes yang menjelaskan fitur, batas cache read-only, preview foto lokal, mock memory, dan pengembangan lanjut. README memuat run, test, build, base URL, serta cara menyiapkan key tanpa credential.

Langkah 6: simpan source, lockfile, hasil pengujian, log build, fingerprint, dan bukti acceptance. Tag day-3 menunjuk source artefak yang diserahkan. Jangan mengubah source setelah tag tanpa memperbarui identitas handover.

Demo akhir menunjukkan alur pengguna, keputusan arsitektur, satu perbaikan terukur, serta backlog berikutnya.

```
git status
git add .
git diff --cached
git commit -m "Complete day 3 quality and release"
git tag day-3
```

Artefak	Periksa
Source/lockfile	Tag dan versi sesuai
Tests/profile	Hasil serta bukti tersedia
APK/AAB	Version dan signer diketahui
Smoke	Device dan hasil nyata
Handover	README, notes, blocker/PIC

A1. Merakit contoh UI lokal

Lampiran berikut memuat source contoh yang lebih lengkap untuk tahap UI lokal. Buat proyek dpl_inspection, lalu tempatkan file pada path yang disebutkan. Semua potongan sebuah file disambung berurutan; label “bagian” dan nomor halaman tidak menjadi bagian kode.

Urutan file adalah validators.dart, models/inspection.dart, inspection_form.dart, lalu main.dart. Import relatif pada contoh sudah mengikuti susunan tersebut. Contoh menggunakan SDK modern yang mendukung initialValue pada DropdownButtonFormField; periksa analyzer jika lingkungan kelas memakai versi lebih lama.

UI contoh menyediakan daftar, detail, form, serta branching loading/empty/error. Loader awal hanya mengembalikan satu record normal. Praktikum meminta peserta mengganti loader dengan data kosong atau error terkontrol untuk mendemonstrasikan semua state.

Source ini sengaja menjadi titik awal hari pertama. Integrasi API, session, cache, dan foto dikerjakan melalui praktikum; tidak diklaim telah terpasang dalam listing UI lokal ini. Mock Python pada lampiran berikutnya berdiri sendiri dan menyediakan kontrak server untuk hari kedua.

Setelah merakit, jalankan format dan analyze lalu run pada Android. Perbaiki error pertama dengan melihat path, import, dan API SDK. Catat hasil di lembar pengujian karena Flutter belum dijalankan di lingkungan penyusunan buku.

```
flutter create --platforms=android dpl_inspection
cd dpl_inspection
# Salin file lib sesuai path pada lampiran.
flutter pub get
dart format lib
flutter analyze
flutter run
```

Validator input — 1/1

lib/validators.dart

Fungsi validasi dipakai oleh form serta parser agar aturan konsisten. Null berarti input valid. Uji batas setelah trim sebelum menghubungkan UI.

Baris 1–18. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
String? validateRequired(String? raw, String label, int max) {
  final value = (raw ?? '').trim();
  if (value.isEmpty) return '$label wajib diisi';
  if (value.length > max) return 'Maksimal $max karakter';
  return null;
}

String? validateNote(String condition, String raw) {
  final value = raw.trim();
  if (!['good', 'needs_action'].contains(condition)) {
    return 'Kondisi tidak dikenal';
  }
  if (value.length > 500) return 'Maksimal 500 karakter';
  if (condition == 'needs_action' && value.length < 10) {
    return 'Catatan temuan minimal 10 karakter';
  }
  return null;
}
```

Model dan JSON — 1/2

lib/models/inspection.dart

Model memeriksa field wajib, condition, note, dan timestamp dengan zona. FormatException menunjukkan kontrak tidak valid. Data cache menggunakan toJson yang menghasilkan UTC.

Baris 1–29. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
import '../validators.dart';

class Inspection {
  final String id, assetCode, location, condition, note;
  final DateTime createdAt;
  const Inspection({
    required this.id, required this.assetCode,
    required this.location, required this.condition,
    required this.note, required this.createdAt,
  });

  factory Inspection.fromJson(Map<String, dynamic> j) {
    String requiredString(String key) {
      final v = j[key];
      if (v is! String || v.trim().isEmpty) {
        throw FormatException('Field $key invalid');
      }
      return v;
    }
    final id = requiredString('id');
    final asset = requiredString('assetCode');
    final location = requiredString('location');
    final condition = requiredString('condition');
    final rawNote = j['note'];
    if (rawNote != null && rawNote is! String) {
      throw const FormatException('Note invalid');
    }
    final note = (rawNote as String?) ?? '';
    if (validateRequired(asset, 'Kode aset', 30) != null ||
```

Model dan JSON — 2/2

lib/models/inspection.dart

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 30–54. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
        validateRequired(location, 'Lokasi', 80) != null ||
        validateNote(condition, note) != null) {
      throw const FormatException('Aturan record invalid');
    }
    final rawTime = requiredString('createdAt');
    final hasZone = RegExp(r'(Z|[+-]\d{2}:?\d{2})$',
        caseSensitive: false).hasMatch(rawTime);
    final time = DateTime.tryParse(rawTime);
    if (!hasZone || time == null) {
      throw const FormatException('Timestamp invalid');
    }
    return Inspection(
      id: id, assetCode: asset.trim(),
      location: location.trim(), condition: condition,
      note: note.trim(), createdAt: time.toUtc(),
    );
  }

  Map<String, dynamic> toJson() => {
    'id': id, 'assetCode': assetCode,
    'location': location, 'condition': condition,
    'note': note,
    'createdAt': createdAt.toUtc().toIso8601String(),
  };
}
```

Form inspeksi lokal — 1/4

lib/inspection_form.dart

Form memiliki controller yang dibuat sekali dan dilepas melalui dispose. Save lokal mengembalikan Inspection kepada daftar; kode ini belum melakukan POST API.

Baris 1–29. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
import 'package:flutter/material.dart';
import 'models/inspection.dart';
import 'validators.dart';

class InspectionForm extends StatefulWidget {
  const InspectionForm({super.key});
  @override
  State<InspectionForm> createState() => _FormState();
}

class _FormState extends State<InspectionForm> {
  final formKey = GlobalKey<FormState>();
  final asset = TextEditingController();
  final location = TextEditingController();
  final note = TextEditingController();
  String condition = 'good';

  @override
  void dispose() {
    asset.dispose();
    location.dispose();
    note.dispose();
    super.dispose();
  }

  void save() {
    if (!formKey.currentState!.validate()) return;
    final now = DateTime.now().toUtc();
    Navigator.of(context).pop(Inspection(
```

Form inspeksi lokal — 2/4

lib/inspection_form.dart

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 30–58. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
      id: 'local-${now.microsecondsSinceEpoch}',
      assetCode: asset.text.trim(),
      location: location.text.trim(),
      condition: condition, note: note.text.trim(),
      createdAt: now,
    ));
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Tambah inspeksi')),
    body: SafeArea(child: SingleChildScrollView(
      padding: const EdgeInsets.all(16),
      child: Form(key: formKey, child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          TextFormField(
            key: const Key('assetField'), controller: asset,
            decoration: const InputDecoration(
              labelText: 'Kode aset',
            ),
            validator: (v) =>
                validateRequired(v, 'Kode aset', 30),
          ),
          TextFormField(
            key: const Key('locationField'),
            controller: location,
            decoration: const InputDecoration(
              labelText: 'Lokasi',
```

Form inspeksi lokal — 3/4

lib/inspection_form.dart

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 59–87. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
            ),
            validator: (v) =>
                validateRequired(v, 'Lokasi', 80),
          ),
          const SizedBox(height: 16),
          DropdownButtonFormField<String>(
            initialValue: condition,
            decoration: const InputDecoration(
              labelText: 'Kondisi',
            ),
            items: const [
              DropdownMenuItem(
                value: 'good', child: Text('Baik'),
              ),
              DropdownMenuItem(value: 'needs_action',
                child: Text('Perlu tindak lanjut')),
            ],
            onChanged: (v) {
              if (v != null) setState(() => condition = v);
            },
          ),
          TextFormField(
            controller: note, minLines: 3, maxLines: 6,
            decoration: const InputDecoration(
              labelText: 'Catatan',
            ),
            validator: (v) => validateNote(condition, v ?? ''),
          ),
          const SizedBox(height: 24),
```

Form inspeksi lokal — 4/4

lib/inspection_form.dart

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 88–99. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
          FilledButton(
            key: const Key('saveButton'),
            onPressed: save, child: const Text('Simpan'),
          ),
          TextButton(onPressed: () =>
              Navigator.of(context).pop(),
            child: const Text('Batal')),
        ],
      )),
    )),
  );
}
```

Aplikasi daftar dan detail — 1/5

lib/main.dart

Entry point merangkai theme, daftar, detail, dan hasil form. Loader contoh mengembalikan data lokal. Ubah loader terkontrol untuk praktik empty dan error.

Baris 1–29. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
import 'package:flutter/material.dart';
import 'models/inspection.dart';
import 'inspection_form.dart';

void main() => runApp(MaterialApp(
  theme: ThemeData(useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal)),
  home: const InspectionListScreen(),
));

class InspectionListScreen extends StatefulWidget {
  const InspectionListScreen({super.key});
  @override
  State<InspectionListScreen> createState() => _ListState();
}

class _ListState extends State<InspectionListScreen> {
  List<Inspection> items = [];
  bool loading = false;
  String? error;

  @override
  void initState() {
    super.initState();
    load();
  }

  Future<void> load() async {
    if (loading) return;
```

Aplikasi daftar dan detail — 2/5

lib/main.dart

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 30–58. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
    setState(() { loading = true; error = null; });
    try {
      await Future<void>.delayed(
        const Duration(milliseconds: 600));
      final result = [Inspection(
        id: 'local-1', assetCode: 'PMP-001',
        location: 'Workshop A', condition: 'good',
        note: '', createdAt: DateTime.now().toUtc(),
      )];
      if (!mounted) return;
      setState(() => items = result);
    } catch (_) {
      if (!mounted) return;
      setState(() => error = 'Gagal memuat data');
    } finally {
      if (mounted) setState(() => loading = false);
    }
  }

  Future<void> openForm() async {
    final created = await Navigator.of(context)
      .push<Inspection>(MaterialPageRoute(
        builder: (_) => const InspectionForm()));
    if (!mounted || created == null) return;
    setState(() => items = [created, ...items]);
  }

  Widget body() {
    if (loading) {
```

Aplikasi daftar dan detail — 3/5

lib/main.dart

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 59–87. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
      return const Center(child: CircularProgressIndicator());
    }
    if (error != null) {
      return Center(child: Column(
        mainAxisSize: MainAxisSize.min, children: [
          Text(error!),
          TextButton(onPressed: load,
            child: const Text('Coba lagi')),
        ],
      ));
    }
    if (items.isEmpty) {
      return const Center(child: Text('Belum ada inspeksi'));
    }
    return ListView.builder(
      itemCount: items.length,
      itemBuilder: (context, index) {
        final item = items[index];
        return ListTile(
          key: ValueKey(item.id), title: Text(item.assetCode),
          subtitle: Text(item.location),
          trailing: Text(item.condition == 'good'
              ? 'Baik' : 'Temuan'),
          onTap: () => Navigator.of(context).push(
            MaterialPageRoute(builder: (_) =>
              InspectionDetail(item: item))),
        );
      },
    );
```

Aplikasi daftar dan detail — 4/5

lib/main.dart

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 88–116. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Daftar inspeksi')),
    body: SafeArea(child: body()),
    floatingActionButton: FloatingActionButton(
      onPressed: openForm, tooltip: 'Tambah inspeksi',
      child: const Icon(Icons.add),
    ),
  );
}

class InspectionDetail extends StatelessWidget {
  final Inspection item;
  const InspectionDetail({super.key, required this.item});
  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: Text(item.assetCode)),
    body: SafeArea(child: SingleChildScrollView(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text('ID: ${item.id}'),
          Text('Lokasi: ${item.location}'),
          Text('Kondisi: ${item.condition == "good"
              ? "Baik" : "Perlu tindak lanjut"}'),
          Text('Catatan: ${item.note.isEmpty ? "-" : item.note}'),
```

Aplikasi daftar dan detail — 5/5

lib/main.dart

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 117–122. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
          Text('Waktu: ${item.createdAt.toLocal()}'),
        ],
      ),
    )),
  );
}
```

Server mock Python — 1/6

mock_api/mock_api.py

Sambung seluruh bagian ke satu file. Server standard library menggunakan data memory, token demo, pagination sederhana, dan flag untuk simulasi kegagalan.

Baris 1–29. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
"""Mock pembelajaran; data memory, bukan backend produksi."""
import argparse
import json
import threading
import time
from datetime import datetime, timezone
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import parse_qs, urlparse

TOKEN = 'training-demo-token'
LOCK = threading.Lock()
ITEMS = [{
    'id': '1', 'assetCode': 'PMP-001',
    'location': 'Workshop A', 'condition': 'good',
    'note': '', 'createdAt': '2026-10-06T01:30:00Z',
}]

def validate(raw):
    if not isinstance(raw, dict):
        return None, 'Body harus object JSON'
    keys = ['assetCode', 'location', 'condition']
    if any(not isinstance(raw.get(k), str) for k in keys):
        return None, 'Field wajib harus String'
    note = raw.get('note', '')
    if not isinstance(note, str):
        return None, 'Note harus String'
    values = {k: raw[k].strip() for k in keys}
    values['note'] = note.strip()
    if not 1 <= len(values['assetCode']) <= 30:
```

Server mock Python — 2/6

mock_api/mock_api.py

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 30–58. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
        return None, 'Kode aset wajib, maksimal 30'
    if not 1 <= len(values['location']) <= 80:
        return None, 'Lokasi wajib, maksimal 80'
    if values['condition'] not in ['good', 'needs_action']:
        return None, 'Kondisi invalid'
    if len(values['note']) > 500:
        return None, 'Note maksimal 500'
    if (values['condition'] == 'needs_action' and
            len(values['note']) < 10):
        return None, 'Catatan temuan minimal 10 karakter'
    return values, None

class Handler(BaseHTTPRequestHandler):
    def reply(self, status, body, delay=True):
        if delay and self.server.config.delay:
            time.sleep(self.server.config.delay)
        data = json.dumps(body).encode('utf-8')
        try:
            self.send_response(status)
            self.send_header('Content-Type',
                             'application/json; charset=utf-8')
            self.send_header('Content-Length', str(len(data)))
            self.end_headers()
            self.wfile.write(data)
        except (BrokenPipeError, ConnectionResetError):
            pass  # Client mungkin timeout setelah proses selesai.

    def authenticated(self):
        return self.headers.get('Authorization') == f'Bearer {TOKEN}'
```

Server mock Python — 3/6

mock_api/mock_api.py

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 59–87. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
    def read_json(self):
        n = int(self.headers.get('Content-Length', '0'))
        if n > 64 * 1024:
            raise ValueError('Body terlalu besar')
        return json.loads(self.rfile.read(n))

    def do_GET(self):
        parsed = urlparse(self.path)
        if parsed.path == '/health':
            return self.reply(200, {'status': 'ok'}, delay=False)
        if parsed.path != '/inspections':
            return self.reply(404, {'message': 'Tidak ditemukan'})
        if not self.authenticated():
            return self.reply(401, {'message': 'Session invalid'})
        if self.server.config.fail_list:
            return self.reply(500, {'message': 'Simulasi list gagal'})
        try:
            q = parse_qs(parsed.query)
            page = int(q.get('page', ['1'])[0])
            size = int(q.get('per_page', ['50'])[0])
            if page < 1 or not 1 <= size <= 50:
                raise ValueError()
        except ValueError:
            return self.reply(422, {'message': 'Pagination invalid'})
        with LOCK:
            snapshot = [dict(x) for x in ITEMS]
        start = (page - 1) * size
        return self.reply(200, {
```

Server mock Python — 4/6

mock_api/mock_api.py

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 88–116. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
            'items': snapshot[start:start + size],
            'total': len(snapshot), 'page': page,
        })

    def do_POST(self):
        path = urlparse(self.path).path
        if path not in ['/auth/login', '/inspections']:
            return self.reply(404, {'message': 'Tidak ditemukan'})
        try:
            raw = self.read_json()
        except (ValueError, TypeError):
            return self.reply(400, {'message': 'JSON invalid'})
        if path == '/auth/login':
            if (isinstance(raw, dict) and
                raw.get('username') == 'trainer' and
                    raw.get('password') == 'training123'):
                return self.reply(200, {
                    'token': TOKEN,
                    'user': {'id': 'u1', 'name': 'Trainer'},
                })
            return self.reply(401, {'message': 'Login salah'})
        if not self.authenticated():
            return self.reply(401, {'message': 'Session invalid'})
        if self.server.config.fail_create:
            return self.reply(500, {'message': 'Simulasi create gagal'})
        values, error = validate(raw)
        if error:
            return self.reply(422, {'message': error})
        with LOCK:
```

Server mock Python — 5/6

mock_api/mock_api.py

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 117–145. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
            record = dict(values, id=str(len(ITEMS) + 1),
                createdAt=datetime.now(timezone.utc).isoformat()
                    .replace('+00:00', 'Z'))
            ITEMS.insert(0, record)
        # Delay response dilakukan setelah data tersimpan.
        return self.reply(201, record)

    def log_message(self, fmt, *args):
        # Access log tanpa body, password, atau token.
        super().log_message(fmt, *args)

def main():
    p = argparse.ArgumentParser()
    p.add_argument('--port', type=int, default=8000)
    p.add_argument('--delay', type=float, default=0)
    p.add_argument('--fail-list', action='store_true')
    p.add_argument('--fail-create', action='store_true')
    args = p.parse_args()
    if args.delay < 0:
        p.error('delay tidak boleh negatif')
    server = ThreadingHTTPServer(('0.0.0.0', args.port), Handler)
    server.config = args
    print(f'Mock API port {args.port}; Ctrl+C untuk berhenti')
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        pass
    finally:
        server.server_close()
```

Server mock Python — 6/6

mock_api/mock_api.py

Lanjutan file yang sama. Tempel langsung setelah baris terakhir pada bagian sebelumnya; jangan menambahkan import atau pembungkus class baru.

Baris 146–148. Indentasi merupakan bagian kode. Perintah terminal yang terbungkus saat dicetak tetap dijalankan sebagai satu baris.

```
if __name__ == '__main__':
    main()
```

A2. Signing Kotlin DSL — persiapan

Fragmen berikut dipasang pada android/app/build.gradle.kts, dengan import pada awal file dan deklarasi props setelah blok plugins. Pertahankan konfigurasi Flutter serta Android yang sudah ada; jangan mengganti seluruh file dengan fragmen ini.

Contoh memeriksa bahwa key.properties tersedia agar kegagalan build latihan mudah dikenali. Kebijakan ini dapat disesuaikan untuk pipeline tim, tetapi tidak boleh secara diam-diam fallback ke debug signer untuk artefak release.

key.properties tidak dikomit. storeFile merupakan path lokal ke keystore khusus training. Password diisi langsung pada mesin pemilik dan tidak menjadi bagian handover. Pastikan file key benar-benar ada sebelum build.

Bagian berikutnya berada di dalam android. Jika signingConfigs release sudah ada, edit blok tersebut; jangan membuat nama konfigurasi yang sama dua kali.

```
import java.util.Properties
import java.io.FileInputStream

// Setelah blok plugins, sebelum android:
val props = Properties()
val propsFile = rootProject.file("key.properties")
require(propsFile.exists()) {
    "Siapkan android/key.properties untuk latihan"
}
FileInputStream(propsFile).use { props.load(it) }
```

Latihan terarah: Verifikasi path dan alias menggunakan keytool tanpa menampilkan password.

A3. Signing Kotlin DSL — release

Letakkan signingConfigs dan buildTypes di dalam blok android yang sudah ada. Konfigurasi release memakai nilai props yang dimuat pada bagian sebelumnya. Pertahankan namespace, defaultConfig, dan konfigurasi lain yang diperlukan proyek.

storeFile dibaca sebagai path file pada mesin build. Setelah konfigurasi selesai, lakukan build APK release dan verifikasi signature. Keberadaan blok signing saja bukan bukti certificate yang dipakai benar.

Bila file menggunakan Groovy, sesuaikan sintaks berdasarkan dokumentasi Flutter deployment Android. Jangan mencampur create("release") Kotlin DSL dengan property access Groovy dalam file yang sama.

Hasil verifikasi dicatat pada lembar release bersama application ID, version, commit, dan environment. Training key tidak digunakan untuk mengganti signer aplikasi produksi yang sudah terdistribusi.

```
// Di dalam android { ... }
signingConfigs {
    create("release") {
        keyAlias = props.getProperty("keyAlias")
        keyPassword = props.getProperty("keyPassword")
        storeFile = file(props.getProperty("storeFile"))
        storePassword = props.getProperty("storePassword")
    }
}
buildTypes {
    release {
        signingConfig = signingConfigs.getByName("release")
    }
}
```

Latihan terarah: Build, apksigner verify, dan bandingkan SHA-256 certificate dengan keystore training.

A4. Soal pemahaman hari pertama

Jawab dengan contoh dari source atau demo. Gunakan kalimat singkat yang menjelaskan hubungan sebab dan perilaku. Soal ini membantu reviewer melihat apakah peserta memahami keputusan yang telah dipraktikkan.

1. Mengapa load tidak dipanggil dari build? Sebutkan dua gejala yang mungkin terjadi jika aturan ini dilanggar.

2. Apa perbedaan final dan object immutable? Berikan contoh List yang final tetapi masih dapat ditambah.

3. Mengapa input hanya spasi ditolak pada kode aset? Di boundary mana trim diterapkan?

4. Mengapa ListView dalam Column body biasanya membutuhkan Expanded? Apa yang perlu diperiksa jika parent sendiri merupakan scroll view?

5. Siapa pemilik TextEditingController dan kapan resource tersebut dilepas?

6. Mengapa form mengembalikan Inspection kepada daftar? Apa yang terjadi jika kedua layar menambahkan record?

7. Apa perbedaan empty dan error? Berikan aksi pengguna yang sesuai untuk masing-masing.

8. Apa yang dilakukan mounted guard setelah await? Apakah guard tersebut otomatis membatalkan request?

9. Mengapa ID record lebih cocok daripada index sebagai ValueKey?

10. Bagaimana membuktikan bahwa cancel form tidak mengubah daftar?

Latihan terarah: Pilih tiga soal lalu tunjukkan jawaban melalui source. Simpan catatan konsep yang masih perlu diperbaiki.

A5. Soal pemahaman data dan release

1. Bedakan tanggung jawab service, repository, controller, dan UI menggunakan contoh GET daftar inspeksi.

2. Mengapa response 200 dengan JSON invalid bukan empty? Bagaimana parsing error seharusnya terlihat pada UI?

3. Mengapa 401 tidak memakai cache fallback? Apa saja data yang dibersihkan?

4. Apa arti fetchedAt dan mengapa waktunya tidak diganti ketika cache dibaca?

5. Mengapa POST timeout tidak berarti data pasti gagal disimpan? Apa risiko retry otomatis?

6. Apa perbedaan submit guard dan idempotency server?

7. Bagaimana mencegah request session lama menulis cache setelah logout?

8. Mengapa preview foto lokal tidak membuktikan foto persistent atau sudah di-upload?

9. Kapan retrieveLostData diperiksa dan bagaimana pending ID membantu recovery?

10. Perilaku apa yang tidak dapat dibuktikan hanya dengan widget test?

11. Mengapa profiling dilakukan pada mode profile dan kondisi device yang sama?

12. Apa perbedaan APK, AAB, upload key, dan app signing key?

13. Bagaimana membuktikan APK menggunakan training signer yang benar?

14. Jika staging belum siap, hasil mana yang masih dapat diperiksa dan mana yang terhalang prasyarat?

Latihan terarah: Jawab minimal satu soal dengan hasil HTTP, satu dengan trace state, dan satu dengan bukti signature.

A6. Pembahasan hari pertama

1. build dapat dipanggil berulang; load di sana dapat membuat operasi ganda dan hasil tidak berurutan. Load awal dipanggil dari lifecycle atau aksi pengguna.

2. final membatasi assignment variabel, bukan mutation object. final list dapat ditambah bila List mutable. Object unmodifiable atau desain immutable membatasi perubahan lain.

3. trim menghapus spasi tepi; input yang kemudian kosong tidak menunjukkan identitas aset. Validasi dan pembentukan record memakai nilai yang dinormalisasi.

4. ListView perlu viewport tinggi terbatas. Expanded cocok bila Column menerima tinggi terbatas. Dalam scroll vertikal dengan tinggi tak terbatas, pola flex perlu ditinjau ulang.

5. State form yang membuat controller menjadi pemiliknya dan memanggil dispose saat selesai.

6. Daftar memiliki mutation agar satu hasil ditambahkan sekali. Penambahan di kedua layar dapat menghasilkan duplikasi.

7. Empty adalah hasil valid tanpa item, error adalah operasi gagal. Empty menawarkan tambah; error menawarkan recovery yang sesuai.

8. mounted menjaga agar State yang selesai tidak diubah. Ia tidak otomatis membatalkan server request.

9. ID adalah identitas stabil, index hanya posisi yang dapat berubah setelah insert atau filter.

10. Catat jumlah sebelum cancel, tutup form tanpa result, lalu periksa jumlah serta isi tetap. Uji juga form valid untuk membandingkan kedua jalur.

A7. Pembahasan data dan release

Service mengelola transport/status/parsing; repository memilih sumber serta cache; controller mengelola operation state; UI membaca state dan menerima input. JSON invalid adalah pelanggaran kontrak, bukan hasil [] yang valid.

401 berarti session tidak diterima: clear token, snapshot, serta state session lama. fetchedAt menyatakan waktu snapshot diperoleh; membarukannya saat baca membuat data lama tampak baru.

POST timeout meninggalkan hasil server belum pasti. Submit guard mencegah operasi paralel pada client, sedangkan idempotency menjamin retry operasi menurut kontrak server. Keduanya berbeda.

Session generation dan ownership mencegah hasil lama di-commit. Penulisan cache juga harus dikendalikan agar clear logout tidak dibatalkan side effect terlambat. Controller dispose saja tidak cukup bila repository tetap menulis storage.

Foto picker dapat berada di cache sementara dan belum dikirim ke API. Recovery diperiksa saat startup Android, lalu hasil dipetakan menggunakan pending ID, bukan detail yang kebetulan terbuka.

Widget test tidak membuktikan kamera, permission, TLS release, atau signature APK. Profiling memerlukan mode dan kondisi yang sebanding agar dampak perubahan dapat dinilai.

APK dapat diinstal; AAB diproses untuk distribusi. Upload key menandatangani upload, app signing key menandatangani app yang didistribusikan melalui Play App Signing. Bandingkan fingerprint pada konteks yang tepat.

Bila staging belum siap, build serta signature tetap diperiksa. Smoke API release diberi status terhalang prasyarat dengan PIC; status tersebut tidak diganti menjadi lulus.

A8. Catatan toolchain dan checkpoint

Lembar ini diisi peserta sebelum praktik dan diperbarui saat handover. Jangan mencatat password, token, atau isi private key. Tuliskan identifier serta hasil pemeriksaan yang diperlukan untuk mengulangi environment.

Status yang digunakan: Lulus, Perlu perbaikan, dan Terhalang prasyarat. Bila status bukan Lulus, sebutkan gejala, PIC, serta tindakan berikutnya. Pemeriksaan toolchain selesai ketika template berjalan, bukan hanya ketika SDK terunduh.

Checkpoint menunjuk commit yang telah diperiksa. Nama tag yang benar tidak menjamin source lulus; lampirkan bukti yang terkait dengan tag tersebut. Setiap perubahan setelah checkpoint perlu commit dan pengujian ulang sesuai dampaknya.

Informasi	Isi peserta
Nama / kelompok	_____
Flutter / Dart	_____
JDK / Android SDK	_____
Device / Android version	_____
Base URL debug / staging	_____
Tag day-1 / commit	_____
Tag day-2 / commit	_____
Tag day-3 / commit	_____
Prasyarat belum siap / PIC	_____

Latihan terarah: Lampirkan hasil doctor, daftar device, dan screenshot run awal pada laporan peserta.

A9. Checklist perilaku inti

Gunakan checklist bersama pasangan reviewer. Setiap kelompok diuji melalui aksi nyata pada aplikasi. Kolom bukti dapat diisi nama screenshot, test, atau log yang aman untuk dibagikan.

Lulus berarti hasil aktual sesuai kontrak. Perlu perbaikan berarti implementasi telah dicoba tetapi perilakunya belum sesuai. Terhalang prasyarat berarti tidak dapat dicoba karena device, staging, atau akun belum tersedia. Jangan menyamakan ketiganya.

Untuk validasi, sertakan data batas. Untuk network, sertakan konfigurasi mock dan status HTTP. Untuk foto, sertakan dua ID record. Untuk release, gunakan artefak APK yang signature-nya sudah diperiksa.

Kelompok skenario	Status / bukti
Daftar dan dua detail	_____
Wajib, trim, batas panjang	_____
needs_action note 9/10	_____
Create sekali dan cancel	_____
Loading/empty/error/retry	_____
Login/restart/401/logout	_____
Cache timestamp/read-only	_____
POST timeout/input tetap	_____
Foto A/B/cancel/recovery	_____
Keyboard/landscape/font	_____
APK release smoke	_____

Latihan terarah: Ulangi skenario yang gagal setelah perbaikan dan simpan hasil aktual yang terbaru.

A10. Log pengujian dan profil

Isi satu baris untuk satu skenario. Data uji yang sama membantu reviewer mengulang hasil. Hindari angka performa yang tidak diukur atau label lulus yang hanya berdasarkan perkiraan.

Untuk profiling, simpan trace sebelum dan sesudah serta commit perubahan. Device, refresh rate, data, mode build, dan skenario dipertahankan. Catatan dampak dapat bersifat kualitatif bila metrik numerik belum tersedia, selama sumber buktinya jelas.

Untuk test, tulis assertion perilaku yang dipakai. Jika sengaja merusak aturan untuk demonstrasi, simpan catatan fail dan pastikan source sudah dipulihkan sebelum handover. Blocker mempunyai PIC dan langkah berikutnya.

Catatan	Isi
Skenario / data uji	_____
Commit / tag	_____
Device / refresh rate	_____
Mode / jumlah record	_____
Aksi pengguna	_____
Hasil diharapkan	_____
Hasil aktual / status	_____
Bukti sebelum	_____
Perubahan satu sebab	_____
Bukti sesudah	_____
Dampak / batas / PIC	_____

A11. Template release notes

Isi template sesuai hasil aktual. Nama versi dan nomor build harus sama dengan artefak. Bila ada pemeriksaan yang belum dilakukan, tulis status serta prasyaratnya agar tim penerima tidak mengira seluruh jalur telah lulus.

Ringkasan fitur: daftar/detail, create dengan validasi, login, GET/POST, search/filter, error/retry, cache snapshot read-only, dan preview foto lokal. Hapus item yang belum benar-benar tersedia pada implementasi peserta.

Batas yang harus dinyatakan: foto belum persistent atau upload; cache tidak menerima transaksi offline; mock memakai data memory; iOS hanya overview. Jika tim menambah implementasi lanjutan, ganti batas sesuai bukti test baru.

Tindak lanjut diprioritaskan berdasarkan kebutuhan: backend nyata, otorisasi, persistensi attachment, idempotency, monitoring, dan distribusi. Tetapkan PIC setiap pekerjaan, bukan hanya daftar harapan.

Bagian	Isi release notes
Version / build / tanggal	_____
Commit / tag / environment	_____
Application ID / signer	_____
Fitur yang tersedia	_____
Analyze/test/device results	_____
Bug yang diperbaiki	_____
Batas dan risiko tersisa	_____
Blocker / PIC / langkah	_____
Lokasi source/artefak/bukti	_____

A12. Istilah yang dipakai dalam buku

Istilah teknis dipakai konsisten supaya diskusi tim tidak mengaburkan sumber masalah. Controller pada buku adalah pemilik state tampilan; ia bukan TextEditingController dan bukan server controller.

Session scope berarti rentang hidup dependency yang dipakai satu session aktif. Ownership menentukan pihak yang membuat, menyimpan, serta melepas resource. State valid memerlukan data yang masih relevan terhadap session dan record tujuan.

Istilah	Makna pada latihan
Widget tree	Hubungan parent-child tampilan
Constraint	Batas ukuran yang diberikan parent
State	Data yang menentukan tampilan saat ini
Future	Hasil operasi yang belum selesai
Repository	Kebijakan sumber data/cache
Service	Transport HTTP/status/parsing
Snapshot	Salinan data pada waktu tertentu
Read-only	Baca tanpa create atau mutation
Idempotency	Retry operasi tidak menggandakan hasil
Signer	Identitas certificate penandatangan
Smoke test	Pemeriksaan alur utama pada artefak nyata
Checkpoint	Source yang ditandai setelah diperiksa

REFERENSI

Sumber materi dan rujukan resmi

Dasar program: 00_Silabus_Flutter_DPL_3_Hari(1).pdf; Hari_1_Flutter_Professional(2).pptx; Hari_2_Flutter_Professional(2).pptx; dan Hari_3_Flutter_Professional(2).pptx. Jadwal, studi kasus, acceptance, serta batas Android/iOS mengikuti dokumen tersebut. Logo diekstrak dari slide sampul dan dipertahankan proporsinya.

Pengembangan editorial meliputi penjelasan konsep, urutan praktikum, parsing defensif, penanganan race session, startup recovery foto, mock Python mandiri, contoh UI lokal, soal, serta lembar kerja. Tambahan ini ditujukan untuk pembelajaran cetak dan tidak mengubah persyaratan komersial.

Rujukan resmi berikut diperiksa pada 6 Oktober 2026 untuk konfirmasi API dan alur platform. Nomor versi SDK/paket tetap dicatat di kelas. Dokumen online dapat berubah; gunakan halaman yang sesuai versi dependency terpasang.

Flutter UI/layout: docs.flutter.dev/ui/layout/constraints
Form: docs.flutter.dev/cookbook/forms/validation
Navigation: docs.flutter.dev/cookbook/navigation/returning-data
Dart async: dart.dev/language/async
Architecture: docs.flutter.dev/app-architecture/guide
Testing: docs.flutter.dev/testing/overview
Android release: docs.flutter.dev/deployment/android
Android signing: developer.android.com/studio/publish/app-signing
Paket: pub.dev/packages/http; pub.dev/packages/image_picker; pub.dev/packages/shared_preferences; pub.dev/packages/flutter_secure_storage.

Kode contoh ditulis sebagai bahan latihan. Smoke HTTP mock Python diperiksa pada lingkungan penyusunan; Flutter run/test/build dan perangkat belum dijalankan di lingkungan ini. Lembar acceptance digunakan untuk membuktikan hasil implementasi kelas.

Bukti Aplikasi pada iPhone 16 Simulator

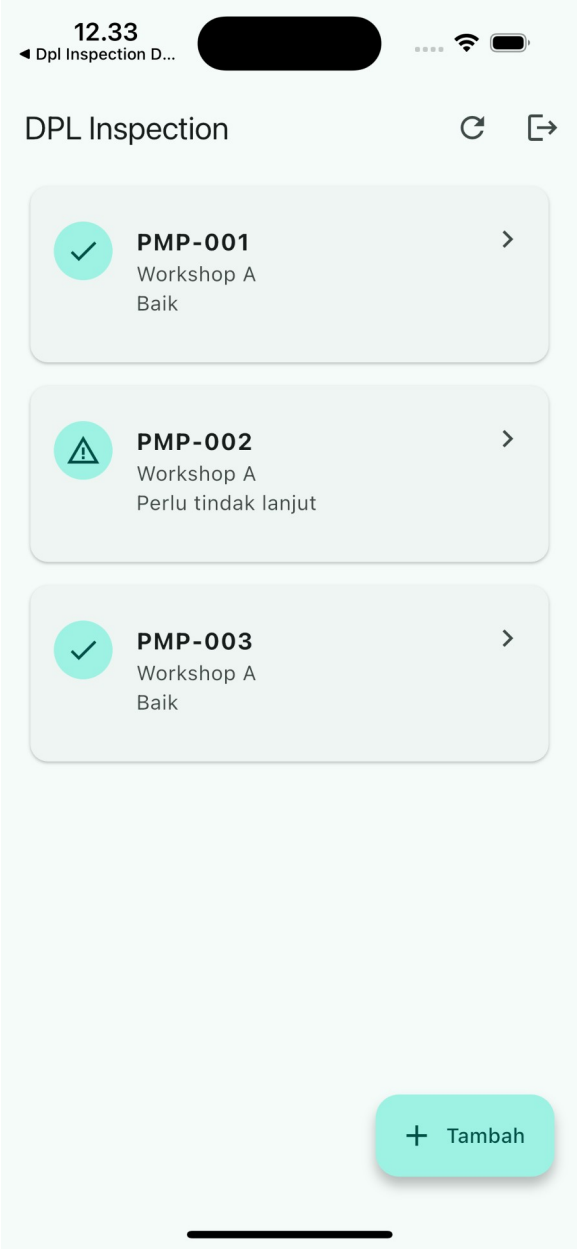
Seluruh gambar pada lampiran ini diambil langsung dari iPhone 16 Simulator iOS 18.0, bukan mockup. Toolchain verifikasi: Flutter 3.44.8 stable, Dart 3.12.2, DevTools 2.57.0, dan Xcode 26.5.

Hari 1 — UI, daftar, dan state lokal



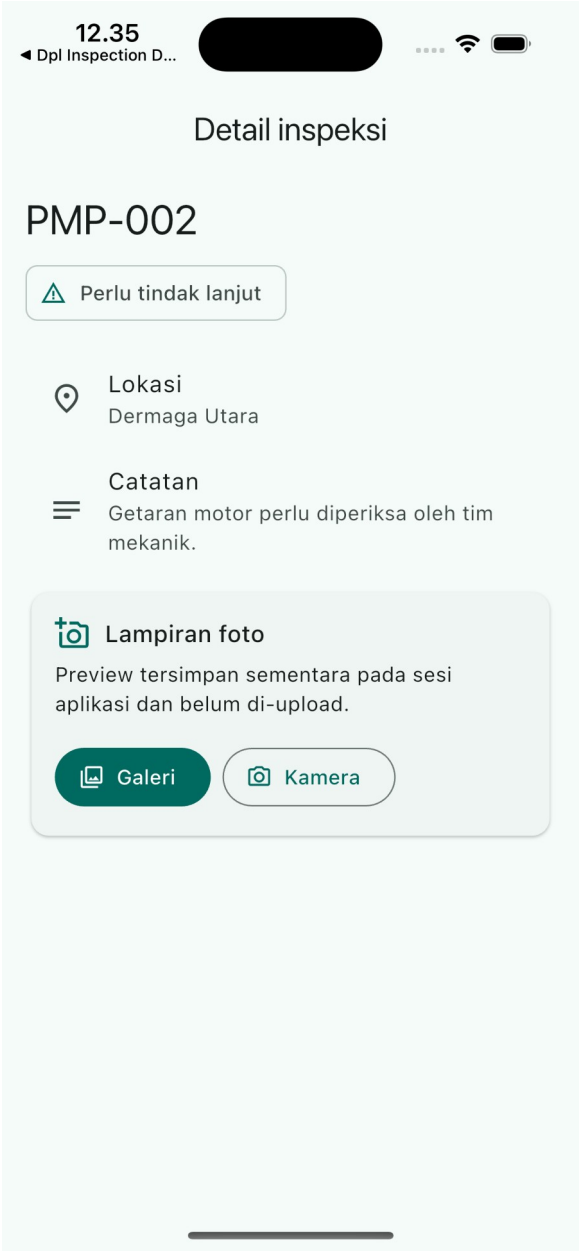
Daftar inspeksi berjalan pada iPhone 16 Simulator.

Hari 2 — REST API, session, dan cache



Data mock API berhasil dimuat setelah login otomatis khusus dokumentasi.

Hari 3 — detail dan lampiran foto



Panel Galeri/Kamera pada detail inspeksi; kamera divalidasi pada perangkat fisik.